Universidade Federal de Minas Gerais
Escola de Engenharia
Curso de Graduação em Engenharia de Sistemas

Desenvolvimento de um Algoritmo Heurístico para Otimização de Leitos Hospitalares

Gustavo Ferreira Tavares

Orientadora: Profa. Gabriela Nunes Lopes
Co-orientadora: Profa. Luiza Bernardes Real

Belo Horizonte, Novembro de 2026

Resumo

A alocação de pacientes a leitos é uma das decisões mais críticas da gestão hospitalar, pois afeta a qualidade do atendimento, o conforto dos pacientes e o uso dos recursos disponíveis. Este trabalho trata o Problema de Alocação de Leitos em sua versão voltada ao planejamento tático do censo hospitalar, na qual pacientes com datas de admissão e de alta conhecidas devem ser alocados aos quartos de um hospital ao longo de um horizonte, minimizando uma soma ponderada de penalidades por especialidade inadequada, transferências e quartos ocupados por pacientes de gêneros distintos, sem exceder a capacidade dos quartos. O problema é formulado como um modelo de Programação Linear Inteira, resolvido pelo *solver* Gurobi, e abordado por um Algoritmo Memético que combina um Algoritmo Genético com busca local por descida em vizinhança variável (VND) e trata a capacidade como restrição penalizada na busca local, com uma etapa de reparo que garante soluções viáveis. As duas abordagens são comparadas, sob o mesmo teto de tempo, em 40 conjuntos de instâncias sintéticas de quatro portes, com 100 a 6400 pacientes. Na instância de menor porte, o *solver* obtém as melhores soluções em nove de dez conjuntos, e a heurística fica, em média, 1,6% acima delas e, em média, no máximo 8,2% acima do ótimo. A partir de 400 pacientes, o *solver* termina o tempo disponível longe do ótimo, e a heurística obtém soluções viáveis com valor da função objetivo de 55% a 96% menor. Um limite inferior combinatório, derivado diretamente dos dados, mostra que a heurística fica a no máximo 0,3 ponto percentual do mínimo de alocações em especialidade inadequada imposto pelos dados. A heurística é também estável, com coeficiente de variação médio de Z inferior a 2% entre execuções. Nas instâncias maiores, a etapa de reparo, além de garantir a viabilidade, reduziu o valor da função objetivo em 6% a 14% em relação à mesma heurística sem reparo. Os resultados indicam que a abordagem heurística é adequada ao planejamento do censo em hospitais de grande porte, nos quais o método exato não é viável no tempo disponível.

Palavras-chave: Alocação de leitos. Algoritmo memético. Programação linear inteira. Planejamento hospitalar. Otimização combinatória.

Abstract

Assigning patients to beds is one of the most critical decisions in hospital management, as it affects the quality of care, patient comfort and the use of available resources. This work addresses the Patient-Bed Allocation problem in its version aimed at the tactical planning of the hospital census, in which patients with known admission and discharge dates must be assigned to the rooms of a hospital over a planning horizon, minimizing a weighted sum of penalties for specialty mismatch, transfers and rooms occupied by patients of different genders, without exceeding room capacity. The problem is formulated as an Integer Linear Programming model, solved with the Gurobi solver, and tackled by a Memetic Algorithm that combines a Genetic Algorithm with Variable Neighborhood Descent (VND) local search and treats capacity as a penalized constraint in the local search, with a repair step that guarantees feasible solutions. Both approaches are compared, under the same time limit, on 40 sets of synthetic instances of four sizes, with 100 to 6400 patients. On the smallest instance, the solver obtains the best solutions in nine of ten sets, and the heuristic is, on average, 1.6% above them and, on average, at most 8.2% above the optimum. From 400 patients onward, the solver ends the available time far from the optimum, and the heuristic finds feasible solutions with objective function values 55% to 96% lower. A combinatorial lower bound, derived directly from the data, shows that the heuristic stays within 0.3 percentage points of the minimum number of specialty mismatches imposed by the data. The heuristic is also stable, with a mean coefficient of variation of Z below 2% across runs. On the larger instances, besides guaranteeing feasibility, the repair step reduced the objective function value by 6% to 14% compared with the same heuristic without repair. The results indicate that the heuristic approach is suitable for census planning in large hospitals, where the exact method is not viable within the available time.

Keywords: Patient-bed allocation. Memetic algorithm. Integer linear programming. Hospital planning. Combinatorial optimization.

Lista de Figuras

3.1 Calibração do peso de capacidade W_{cap} — instância média 37

4.1 Dispersão de Z por conjunto — instância pequena 41
4.2 Convergência da heurística nos conjuntos 1 a 6 — instância pequena 42
4.3 Convergência da heurística nos conjuntos 7 a 10 — instância pequena . . . 42
4.4 Dispersão de Z por conjunto — instância média 44
4.5 Convergência da heurística nos conjuntos 1 a 6 — instância média 45
4.6 Convergência da heurística nos conjuntos 7 a 10 — instância média 45
4.7 Dispersão de Z por conjunto — instância grande 47
4.8 Convergência da heurística nos conjuntos 1 a 6 — instância grande 48
4.9 Convergência da heurística nos conjuntos 7 a 10 — instância grande 48
4.10 Dispersão de Z por conjunto — instância muito grande 50
4.11 Convergência da heurística nos conjuntos 1 a 6 — instância muito grande . 51
4.12 Convergência da heurística nos conjuntos 7 a 10 — instância muito grande . 51

Lista de Tabelas

3.1 Características das instâncias geradas 21
3.2 Pesos adotados para os critérios da função objetivo 22
3.3 Parâmetros do Algoritmo Memético 29
3.4 Configuração experimental por porte de instância 35
3.5 Testes de calibração dos parâmetros do Algoritmo Memético 36

4.1 Limites do valor ótimo de Z por conjunto — instância pequena 40
4.2 Resultados da heurística por conjunto — instância pequena 41
4.3 Limites do valor ótimo de Z por conjunto — instância média 43
4.4 Resultados da heurística por conjunto — instância média 44
4.5 Limites do valor ótimo de Z por conjunto — instância grande 46
4.6 Resultados da heurística por conjunto — instância grande 47
4.7 Limites do valor ótimo de Z por conjunto — instância muito grande 49
4.8 Resultados da heurística por conjunto — instância muito grande 50
4.9 Viabilidade das soluções heurísticas por porte 52
4.10 Esforço computacional por porte (média por execução) 53
4.11 Especialidade inadequada por porte 54
4.12 Transferências por porte 55
4.13 Quartos-dia mistos por porte 56
4.14 Função objetivo por paciente-dia e mínimo estrutural por porte 57

Sumário

1 Introdução **7**

2 Revisão Bibliográfica **10**
2.1 Gestão de leitos: problemas similares na literatura 10
2.2 Classificação do problema 11
2.2.1 Complexidade computacional: o desafio NP-difícil 11
2.2.2 Modelagens na literatura 12
2.3 Meta-heurísticas 13
2.4 Combinação de algoritmos 14
2.5 Tratamento de restrições 14
2.6 Etapa de decisão 15
2.6.1 Tomadores de decisão 15
2.7 Fechamento da revisão e definição da arquitetura 15

3 Metodologia **18**
3.1 Descrição do problema 18
3.1.1 Parâmetros, variáveis e discretização do tempo 18
3.1.2 Restrições 19
3.1.3 Pesos e abordagem multiobjetivo 19
3.1.4 Dados e instâncias de teste 20
3.1.5 Gerador de instâncias 20
3.2 Modelagem matemática 21
3.2.1 Conjuntos e parâmetros 22
3.2.2 Variáveis de decisão 23
3.2.3 Função objetivo 23
3.2.4 Restrições 23
3.2.5 Domínio das variáveis 24
3.3 Algoritmo Memético 24
3.3.1 Representação da solução 24
3.3.2 Solução inicial 25
3.3.3 Violação de capacidade e *fitness* 26
3.3.4 Estruturas de vizinhança e busca local 26
3.3.5 Operadores do Algoritmo Genético 28
3.3.6 Etapa de reparo da capacidade 29
3.4 Implementação 30
3.4.1 Avaliação procedural e simplificação do modelo 30

3.4.2 Ferramentas e ambiente de execução 31
3.5 Métricas para avaliação de desempenho 31
3.6 Protocolo experimental 34
3.6.1 Calibração de parâmetros 35

4 Resultados **39**
4.1 Comparação entre o Gurobi e a heurística 39
4.1.1 Instância pequena 39
4.1.2 Instância média 43
4.1.3 Instância grande 46
4.1.4 Instância muito grande 49
4.1.5 Viabilidade e custo do reparo 52
4.1.6 Esforço computacional 53
4.2 Discussão das métricas operacionais 53
4.2.1 Especialidade 54
4.2.2 Transferências 55
4.2.3 Quartos mistos 56
4.2.4 Comparação entre portes 56

5 Conclusão **58**
5.1 Limitações 59
5.2 Trabalhos futuros 60

Referências **61**

Capítulo 1

Introdução

Hospitais são instituições fundamentais para a manutenção da saúde pública, enfrentando uma demanda crescente por seus serviços, impulsionada por fatores como o aumento e o envelhecimento populacional. Contudo, essa demanda ocorre em um cenário de alta pressão por eficiência, devido aos custos ascendentes dos serviços de saúde e à limitação de recursos financeiros e físicos [1, 2]. Nesse contexto, a gestão eficiente dos recursos hospitalares não é apenas uma meta operacional, mas uma necessidade estratégica para garantir a sustentabilidade do sistema e a qualidade do atendimento ao paciente [1, 3].

Um dos recursos mais críticos e, frequentemente, o gargalo das operações hospitalares é a disponibilidade de leitos de internação. A gestão ineficiente de leitos gera impactos em cascata por todo o hospital, sendo apontada como uma das principais causas da superlotação nos serviços de emergência, onde pacientes já admitidos permanecem à espera de um leito de internação. Essa superlotação resulta em longos tempos de espera para pacientes admitidos, desvio de ambulâncias para outras unidades e, consequentemente, compromete a segurança e a qualidade do cuidado [1, 3]. No Brasil, onde o Sistema Único de Saúde (SUS) enfrenta desafios históricos de subfinanciamento [4] e de superlotação dos serviços de emergência, associada principalmente à falta de leitos de internação [5], a otimização do uso dos leitos torna-se ainda mais crucial.

A complexidade do problema é acentuada pela natureza estocástica do ambiente hospitalar, marcada pela incerteza nas chegadas de pacientes, especialmente os de emergência, e pela variabilidade no tempo de permanência (LOS - *Length of Stay*) de cada paciente [6, 3, 2]. Além disso, a alocação de pacientes a leitos transcende uma simples otimização de ocupação. Trata-se de um desafio multiobjetivo que deve conciliar interesses conflitantes de múltiplos *stakeholders*, como pacientes, profissionais da saúde e a administração hospitalar [2]. Este trabalho, contudo, trata a versão determinística do problema, na qual as datas de admissão e de alta de cada paciente são conhecidas.

Do ponto de vista do paciente, busca-se adequação clínica e conforto, como a minimização de transferências internas [2]. Os profissionais da saúde e a administração têm ainda interesses próprios, como o equilíbrio da carga de trabalho das equipes e a utilização eficiente dos recursos [7, 8, 2]. Dentre os critérios possíveis, este trabalho considera a adequação da especialidade clínica, as transferências, a separação de pacientes de gêneros distintos e a capacidade dos quartos.

Para enfrentar esse desafio, diversas abordagens de Pesquisa Operacional, como programação matemática, simulação e modelos baseados em teoria das filas, têm sido propostas

[1, 3]. Métricas como taxa de ocupação, intervalo de substituição entre pacientes e tempo médio de permanência são frequentemente utilizadas na literatura para avaliar a eficiência da alocação de leitos [3]. Neste trabalho, a taxa de ocupação é usada para caracterizar as instâncias, e a qualidade das soluções é avaliada pelas penalidades do modelo proposto. No entanto, o Problema de Alocação de Leitos (PBA - *Patient-Bed Allocation* ou PAS - *Patient Admission Scheduling*) é reconhecido por sua alta complexidade computacional, sendo classificado como NP-difícil [6, 8, 2]. Como não se conhece algoritmo exato de tempo polinomial para problemas NP-difíceis, o tempo exigido por métodos exatos, como a programação linear inteira, tende a crescer rapidamente com o tamanho da instância, o que torna inviável, na maioria das vezes, obter a solução ótima para hospitais de grande porte em tempo hábil [6, 8].

Mesmo em sua versão determinística, o problema cresce rapidamente com o porte do hospital, e os métodos exatos deixam de produzir boas soluções em tempo compatível com o planejamento rotineiro do censo hospitalar [6, 8]. Essa lacuna justifica o desenvolvimento de algoritmos heurísticos e meta-heurísticos, capazes de fornecer soluções de boa qualidade dentro desse prazo, inclusive em instâncias de grande porte [8, 9].

Diante do exposto, o objetivo central deste trabalho é o desenvolvimento de um algoritmo heurístico para o planejamento tático do censo hospitalar, isto é, a alocação de um conjunto conhecido de pacientes, com datas de admissão e de alta previamente definidas, aos quartos do hospital ao longo de um horizonte de planejamento. O problema é formulado como um modelo multiobjetivo de Programação Linear Inteira, sendo a função objetivo tratada como uma soma ponderada de penalidades. A admissão em tempo real, com chegadas de emergência e incerteza no tempo de permanência, está fora do escopo deste trabalho.

O modelo proposto, comum às abordagens heurística e exata, considera variáveis binárias que indicam em qual quarto cada paciente permanece em cada dia de sua internação. Esse modelo tem como objetivo minimizar simultaneamente as alocações em quartos de especialidade inadequada, as transferências durante a internação e a ocupação concomitante de quartos por pacientes de gêneros distintos, respeitando a capacidade de leitos de cada quarto.

A heurística proposta é um Algoritmo Memético, que combina um Algoritmo Genético com busca local por descida em vizinhança variável (VND). Durante a busca local, a capacidade dos quartos é tratada como restrição penalizada, e uma etapa de reparo realoca os pacientes-dia excedentes, de modo que todas as soluções reportadas respeitam a capacidade.

Para contextualizar os resultados obtidos pela heurística, também foi implementada uma abordagem exata, na qual o mesmo modelo é resolvido pelo *solver* comercial Gurobi. Ambas as abordagens são comparadas, sob o mesmo teto de tempo, quanto à qualidade da solução e ao esforço computacional em instâncias sintéticas de diferentes portes, de modo a identificar seu comportamento à medida que o tamanho do cenário simulado aumenta.

Como contribuição adicional, utiliza-se um limite inferior combinatório, derivado diretamente dos dados por uma relaxação na linha da proposta por Ceschia e Schaerf [7], que permite avaliar a distância das soluções ao ótimo mesmo quando o *solver* não fornece um limite útil. Com isso, este trabalho se propõe a criar uma ferramenta de apoio ao planejamento do censo que mantenha soluções de boa qualidade mesmo em instâncias de grande porte, típicas de grandes instituições hospitalares.

Além desta introdução, o trabalho está organizado em quatro capítulos. O Capítulo 2 revisa a literatura sobre o Problema de Alocação de Leitos e as meta-heurísticas empregadas. O Capítulo 3 descreve o problema, o modelo matemático, o Algoritmo Memético, a etapa

de reparo e o protocolo experimental. O Capítulo 4 apresenta e discute os resultados, e o Capítulo 5 reúne as conclusões, as limitações e os trabalhos futuros.

Capítulo 2

Revisão Bibliográfica

2.1 Gestão de leitos: problemas similares na literatura

O gerenciamento da capacidade hospitalar e, em particular, a alocação de leitos, pode ser mapeado para diversas famílias clássicas de problemas de otimização documentadas na literatura de Pesquisa Operacional. À luz da taxonomia de modelos de otimização apresentada por Talbi [9], o Problema de Alocação de Leitos pode ser enquadrado na categoria de Otimização Combinatória, compartilhando características fundamentais com os seguintes problemas clássicos:

- **Problemas de Atribuição (*Assignment Problems*):** Em sua essência, alocar pacientes a quartos é uma extensão do Problema de Atribuição Generalizada (GAP), em que cada objeto é atribuído a um recurso de capacidade limitada, papel que no hospital cabe ao número de leitos de cada quarto. O problema aproxima-se também do Problema de Atribuição Quadrática (QAP), no qual o custo de uma atribuição depende das demais atribuições realizadas [9]. No hospital, a penalidade por quarto misto tem essa natureza, pois não depende de um paciente isolado, mas do par de pacientes de gêneros distintos que compartilham o mesmo quarto no mesmo dia. Outros critérios da literatura, como a proximidade dos pacientes de um mesmo médico, que reduz o tempo de deslocamento da equipe, também dependem de pares de pacientes [2].

- **Problemas de Escalonamento (*Scheduling Problems*):** O tempo de permanência do paciente (LOS) insere a dimensão temporal no problema, e o escalonamento é uma das classes clássicas da otimização combinatória [9]. Brandeau, Sainfort e Pierskalla [1] ressaltam que o planejamento de saúde frequentemente lida com o escalonamento de recursos escassos sob incerteza. Na versão determinística tratada neste trabalho, em que as datas de admissão e de alta são conhecidas [8], a gestão de leitos assemelha-se ao escalonamento de tarefas com intervalos fixos em máquinas paralelas. Os pacientes são as "tarefas", com início e fim predeterminados, e os quartos são as "máquinas", cada uma capaz de atender simultaneamente tantas tarefas quanto seus leitos.

- **Problemas de Empacotamento (*Bin Packing*):** No *Bin Packing*, itens de diferentes tamanhos devem ser acomodados em compartimentos de capacidade finita [9]. No hospital, os quartos atuam como compartimentos cuja capacidade, o número de leitos, não pode ser excedida em nenhum dia, e a duração da internação ou a carga de trabalho de enfermagem exigida pelo paciente pode representar o tamanho do item [10, 9].

Como o número de quartos é fixo, o objetivo não é minimizar o número de compartimentos, como no problema clássico, mas respeitar a capacidade de cada um. A mesma ótica é usada na geração das instâncias deste trabalho, que admite os pacientes por empacotamento (Subseção 3.1.5).

Essas similaridades teóricas são essenciais, pois permitem que a vasta gama de algoritmos já desenvolvidos para esses problemas clássicos na ciência da computação seja adaptada ao ambiente da saúde.

2.2 Classificação do problema

Na literatura internacional, o desafio de acomodar pacientes em unidades de internação é frequentemente referido pela sigla PBA (*Patient-Bed Allocation* - Alocação de Pacientes a Leitos) ou PAS (*Patient Admission Scheduling*).

O PBA não é simplesmente atribuir um leito vago a um paciente. A literatura descreve uma série de critérios, restrições e objetivos conflitantes, que afetam diretamente múltiplos *stakeholders* (pacientes, profissionais da saúde e administração). Entre eles, estão a adequação clínica do quarto e de seus equipamentos [8, 2] e as restrições de gênero, idade, capacidade e isolamento [6, 2]. A literatura trata ainda do conforto do paciente, como perfis semelhantes no mesmo quarto, preferência por quarto individual e ausência de transferências [8], da eficiência dos profissionais, como a proximidade dos pacientes de um mesmo médico e o equilíbrio da carga de trabalho da enfermagem [2], e da eficiência do sistema, como a taxa de ocupação, a espera por leito e a acomodação de pacientes eletivos e de emergência [1, 10].

Este trabalho considera apenas um subconjunto desses critérios:

- **Considerados:** a adequação da especialidade clínica do quarto ao paciente [8, 2], a separação de pacientes de gêneros distintos e o limite de capacidade de cada quarto [6, 2], e a minimização de transferências durante a internação, que causam desconforto ao paciente [8].
- **Não considerados:** faixas etárias, requisitos de isolamento, preferência por quarto individual ou enfermaria, equipamentos específicos de cada leito, proximidade dos pacientes de um mesmo médico, carga de trabalho da enfermagem e o tratamento de chegadas de emergência e da incerteza no tempo de permanência, próprios da admissão em tempo real [6, 2].

2.2.1 Complexidade computacional: o desafio NP-difícil

O PBA é classificado na literatura como um problema NP-difícil (*NP-Hard*) [6, 8, 2]. Segundo a teoria da complexidade detalhada por Talbi [9], para problemas dessa classe não se conhece nenhum algoritmo exato capaz de encontrar a solução ótima garantida em tempo polinomial, de modo que, no pior caso, o tempo exigido pelos métodos exatos cresce exponencialmente com o tamanho da instância.

A essa dificuldade teórica soma-se o tamanho do espaço de busca. Como cada paciente pode, em princípio, ocupar qualquer um dos $|R|$ quartos em cada dia de sua internação,

o número de alocações possíveis chega a $|R|^{\sum_{p \in P} |D_p|}$. Na instância de menor porte deste trabalho, com 10 quartos e cerca de 400 pacientes-dia, isso corresponde a cerca de 10^{400} alocações, o que torna a enumeração impraticável. O tamanho do espaço, por si só, não torna um problema difícil, pois o problema de atribuição clássico também tem um número exponencial de soluções e é resolvido em tempo polinomial. É a combinação das duas características que faz com que, em instâncias de tamanho real, métodos exatos possam não encontrar sequer uma solução viável após uma hora de processamento [8]. Ainda que, no planejamento do censo, as datas de admissão e de alta sejam conhecidas de antemão, a solução precisa ser obtida em tempo compatível com a rotina de planejamento do hospital, o que torna esses métodos impraticáveis nas instâncias maiores.

2.2.2 Modelagens na literatura

Diversos trabalhos na literatura tentaram enfrentar o PBA utilizando *frameworks* matemáticos formais, embora frequentemente esbarrem na barreira da complexidade computacional descrita acima.

As abordagens de Programação Linear Inteira (PLI) e Programação Inteira Mista (MIP) são as mais comuns na tentativa de modelar o problema de forma exata. Demeester et al. [8] desenvolveram um modelo de Programação Inteira para o PBA com o objetivo de minimizar as violações de restrições, como transferências e alocações em departamentos inadequados. Resolvido por um *solver* comercial padrão (ILOG CPLEX), o modelo não encontrou sequer uma solução viável dentro de uma hora de cálculo, enquanto a meta-heurística proposta pelos autores fornecia soluções viáveis mesmo quando interrompida no início da execução.

Ceschia e Schaerf [6] compararam uma abordagem baseada em *Simulated Annealing* com um método exato de Programação Linear Inteira para o PAS. O método exato resolveu apenas instâncias pequenas e médias, de até 250 pacientes, enquanto a meta-heurística resolveu instâncias de até 4000 pacientes em tempo razoável, com qualidade equivalente nas instâncias em que ambos chegaram ao fim. Em trabalho anterior, os mesmos autores [7] calcularam limites inferiores (*lower bounds*) para o PAS a partir da relaxação de parte das restrições e da solução de um problema de emparelhamento de custo mínimo em um grafo bipartido, e suas soluções heurísticas ficaram, em alguns casos, bastante próximas desses limites. O limite inferior combinatório utilizado neste trabalho, conforme a Equação (3.17), segue a mesma ideia de relaxação. Quando se mantém apenas a capacidade de cada dia e o custo de especialidade inadequada, o emparelhamento de pacientes a leitos admite solução em forma fechada, dada pela contagem dos pacientes que excedem, em cada dia, os leitos da especialidade que requerem.

Para lidar com múltiplos objetivos conflitantes (custo, serviço, satisfação), funções objetivo ponderadas têm sido amplamente adotadas. Em um problema relacionado, Beliën e Demeulemeester [10] trataram o nivelamento da ocupação de leitos gerada pelo centro cirúrgico com heurísticas baseadas em Programação Inteira Mista (MIP) e uma meta-heurística, minimizando a falta esperada de leitos. Schäfer et al. [2], por sua vez, formularam uma função de utilidade total, combinando penalidades de médicos, enfermeiros e pacientes. Os autores ressaltam que modelagens analíticas precisas devem ser integradas com lógicas de decisão rápidas para terem valor prático.

2.3 Meta-heurísticas

Para lidar com a intratabilidade de problemas NP-difíceis, como a alocação de leitos (PBA), as meta-heurísticas surgem como a principal alternativa aos métodos exatos. O termo "meta-heurística" define uma metodologia de nível superior que atua como uma estratégia guia para projetar heurísticas subjacentes aplicáveis a uma ampla variedade de problemas de otimização. Embora não garantam a otimalidade global, elas fornecem soluções de alta qualidade ("aceitáveis") em um tempo de processamento razoável [9].

O *design* de qualquer meta-heurística envolve o balanceamento de dois critérios conflitantes, a *exploração* (diversificação), que conduz a busca a regiões ainda não visitadas do espaço de busca, e a *explotação* (intensificação), que concentra a busca em torno das melhores soluções encontradas para gerar resultados ainda melhores [9]. De acordo com suas características de busca, as meta-heurísticas são tipicamente divididas em duas grandes famílias:

- **Meta-heurísticas baseadas em trajetória (*S-metaheuristics*):** Focam na transformação iterativa de uma única solução, sendo altamente orientadas à explotação (intensificação). A forma mais simples é a busca local, que parte de uma solução e a substitui repetidamente por uma solução vizinha melhor, obtida por uma pequena modificação chamada movimento, até alcançar um ótimo local, isto é, uma solução sem nenhum vizinho melhor. A descida em vizinhança variável (*Variable Neighborhood Descent* - VND) generaliza essa ideia ao empregar várias estruturas de vizinhança, examinadas em uma ordem predefinida. Quando uma vizinhança não produz melhora, passa-se à seguinte, e o método termina quando nenhuma delas melhora a solução corrente, que é então um ótimo local em relação a todas as vizinhanças [9]. A Busca Tabu (*Tabu Search* - TS) utiliza estruturas de memória (listas tabu) para registrar o histórico recente da busca, impedindo que o algoritmo visite soluções recém-avaliadas e fique preso em ótimos locais [9]. Demeester et al. [8] aplicaram um algoritmo de Busca Tabu híbrido ao problema de admissão de pacientes, obtendo soluções viáveis em tempos nos quais o método exato não encontrou nenhuma. O *Simulated Annealing* (SA), inspirado no processo de recozimento de metais, utiliza uma função de probabilidade decrescente (temperatura) para aceitar soluções piores no início da busca, evitando ótimos locais e convergindo gradativamente para uma solução refinada [9]. Ceschia e Schaerf [6] utilizaram o SA, com uma estrutura de vizinhança composta, para resolver as versões estática e dinâmica do PAS sob incerteza.

- **Meta-heurísticas baseadas em população (*P-metaheuristics*):** Lidam simultaneamente com um conjunto de soluções (população), sendo naturalmente orientadas à exploração (diversificação) do espaço de busca. Os Algoritmos Genéticos (*Genetic Algorithms* - GA) e outras abordagens evolutivas imitam a evolução biológica e a seleção natural, combinando soluções através de operadores de cruzamento (*crossover*) e mutação para gerar novas "descendências" [9, 11]. A Inteligência de Enxame, como a Otimização por Nuvem de Partículas (*Particle Swarm Optimization* - PSO) e a Otimização por Colônia de Formigas (*Ant Colony Optimization* - ACO), baseia-se no comportamento descentralizado e auto-organizado de animais na natureza [9, 3, 11].

2.4 Combinação de algoritmos

A literatura demonstra que a aplicação de meta-heurísticas puras pode apresentar limitações ao lidar com espaços de busca altamente combinatórios e restritos. A hibridização, ou seja, a combinação de diferentes algoritmos, é uma estratégia adotada para suprir as fraquezas de um método com os pontos fortes de outro [9].

No contexto da saúde e do planejamento de capacidade, diversas abordagens híbridas obtiveram sucesso ao aliar qualidade e velocidade computacional:

- **Híbridos de P-metaheurísticas e S-metaheurísticas:** Uma abordagem clássica (conhecida como Algoritmo Memético) incorpora um algoritmo de busca local dentro de um Algoritmo Genético. Enquanto o GA explora globalmente o espaço de soluções por meio da população, a busca local atua como um operador de "mutação inteligente", refinando cada indivíduo e acelerando a convergência para ótimos locais de alta qualidade [9].

- **Híbridos de Meta-heurísticas e Métodos Exatos:** Métodos baseados em Programação Matemática podem ser usados para resolver subproblemas específicos ou inicializar meta-heurísticas. Beliën e Demeulemeester [10] combinaram heurísticas baseadas em Programação Inteira Mista com uma meta-heurística para nivelar a ocupação de leitos gerada pelo centro cirúrgico.

- **Integração com *Machine Learning*:** Schäfer et al. [2] integraram previsões de aprendizado de máquina a uma *hiper-heurística* para o gerenciamento dinâmico de leitos, contexto que envolve incertezas não tratadas neste trabalho.

2.5 Tratamento de restrições

Em problemas com restrições rígidas, como a capacidade dos quartos, os operadores de uma meta-heurística podem gerar soluções inviáveis. Talbi [9] classifica as estratégias para lidar com essas soluções em cinco grupos:

- **Rejeição:** apenas as soluções viáveis são mantidas, e as inviáveis são descartadas.

- **Penalização:** as soluções inviáveis são aceitas, e a violação é adicionada à função de avaliação como uma penalidade ponderada.

- **Reparo:** uma heurística específica transforma a solução inviável em uma solução viável.

- **Decodificação:** a representação é construída de modo que toda solução decodificada seja viável.

- **Preservação:** a representação e os operadores são projetados para nunca gerar soluções inviáveis.

A rejeição é simples, mas pode restringir excessivamente a busca quando a região viável é estreita ou fragmentada. A penalização permite que a busca atravesse regiões inviáveis,

ao custo de exigir a calibração do peso da penalidade [9]. As estratégias podem também ser combinadas, como neste trabalho, que penaliza a violação da capacidade durante a busca local e aplica uma etapa de reparo à população a cada geração, conforme a Subseção 3.3.6.

2.6 Etapa de decisão

Em problemas de saúde, formular e otimizar um modelo matemático é apenas parte do desafio, e a fase crítica reside na etapa de tomada de decisão. A tomada de decisão é o processo interativo no qual os resultados algorítmicos são avaliados frente a cenários reais, levando à implementação de uma solução prática [9].

No gerenciamento de leitos, a etapa de decisão é de suma importância porque o problema é inerentemente multiobjetivo e sujeito a incertezas [2]. Uma otimização estrita baseada apenas em custos ou taxas de ocupação pode negligenciar fatores humanos relevantes, como o conforto e a segurança do paciente [2]. A etapa de decisão é o momento em que os múltiplos interesses (muitas vezes conflitantes) dos *stakeholders* são balanceados para garantir uma alocação que seja não apenas matematicamente ótima, mas clinicamente e operacionalmente viável.

2.6.1 Tomadores de decisão

Na literatura de otimização multiobjetivo, a interação entre o solucionador (o algoritmo) e o tomador de decisão (gestor hospitalar) pode ocorrer sob três abordagens principais [9]:

- **Abordagem *A Priori*:** O tomador de decisão estabelece suas preferências antes do início da otimização. Isso é frequentemente feito através da agregação das funções objetivo em uma única função de utilidade, na qual cada critério recebe um peso. Essa abordagem é computacionalmente rápida (gera uma única solução e requer apenas uma execução da meta-heurística), mas exige que o gestor conheça antecipadamente o comportamento e a importância de cada critério [9, 2].
- **Abordagem *A Posteriori*:** O algoritmo executa a busca e devolve um conjunto de soluções de compromisso (a Fronteira de Pareto), representando diferentes cenários otimizados (por exemplo, uma solução com menos transferências e outra com menos alocações em especialidade inadequada). Apenas no final o tomador de decisão escolhe a solução desejada. É muito útil quando o cenário não é claro, mas requer alto custo computacional [9].
- **Abordagem Interativa:** Há uma cooperação progressiva entre o gestor e o algoritmo durante a busca. O algoritmo apresenta soluções parciais, o tomador de decisão refina suas preferências, e a busca é redirecionada. É altamente flexível, mas exige envolvimento constante do gestor [9].

2.7 Fechamento da revisão e definição da arquitetura

A gestão de leitos hospitalares foi caracterizada ao longo desta revisão como um desafio multiobjetivo e de complexidade NP-difícil, que, em sua forma geral, envolve ainda incertezas na chegada e no tempo de permanência dos pacientes [6, 8, 3]. Este trabalho trata a

versão determinística do problema, voltada ao planejamento do censo, na qual, ainda assim, os métodos exatos tornam-se inviáveis em tempo hábil nas instâncias de grande porte [6, 8]. Conforme Talbi [9], quando o tempo disponível para a tomada de decisão é limitado, como no planejamento periódico do censo hospitalar, o uso de algoritmos aproximados (heurísticas e meta-heurísticas) é a via mais adequada. Embora essas técnicas não garantam que a solução encontrada seja o ótimo global, elas exploram o espaço de busca de maneira estruturada e, nos trabalhos revisados, obtiveram soluções de boa qualidade em instâncias que os métodos exatos não resolveram no tempo disponível [6, 8]. Assim, a abordagem proposta visa atuar como um sistema de apoio à decisão tática, voltado ao planejamento do censo, oferecendo um equilíbrio entre a qualidade das alocações e um tempo de resposta compatível com esse planejamento.

Diante desse cenário, este trabalho adota um Algoritmo Genético (GA) como motor de otimização global. Por ser baseado em população (*P-metaheuristic*), o GA explora o espaço de busca de forma ampla e mantém, ao longo da execução, uma diversidade de soluções candidatas que podem ser recombinadas [9]. Um GA puro, contudo, não garante bom desempenho no problema. Na comparação feita por Schäfer et al. [2], por exemplo, a hiper-heurística dos autores superou um Algoritmo Genético.

Esse resultado é coerente com a observação de que os GAs, embora eficazes na exploração global (*diversificação*), frequentemente apresentam limitações na fase de explotação (*intensificação*), ou seja, no ajuste fino das soluções para alcançar ótimos locais de alta qualidade [9]. Para suprir essa lacuna e lidar com o alto nível de restrições do PBA, este trabalho propõe uma arquitetura híbrida, denominada Algoritmo Memético, classificada como um híbrido de baixo nível em equipe (*Low-Level Teamwork Hybrid*) [9].

Nesta abordagem, o GA é acoplado a um algoritmo de busca local baseada em múltiplas vizinhanças, sendo o *Variable Neighborhood Descent* (VND) o escolhido. A busca local não substitui o operador de mutação. A mutação do GA é mantida como fonte de diversidade e a busca local atua em seguida como um operador de intensificação aplicado a uma parcela da população a cada geração. Sob uma ótica Lamarckiana [9], o indivíduo refinado substitui o original na população, de modo que as melhorias obtidas pela busca local são herdadas pelas gerações seguintes por meio do cruzamento.

A escolha do VND como agente de busca local acompanha os trabalhos sobre admissão de pacientes, como os de Demeester et al. [8] e Ceschia e Schaerf [7, 6], que empregaram combinações de diferentes movimentos, e não um único tipo, para explorar o espaço de soluções. Ceschia e Schaerf [7], em particular, compararam diferentes combinações de vizinhanças para o PAS. O VND segue a mesma ideia, alternando entre mudança de quarto, troca de pacientes e suas variantes parciais, de modo que uma solução que é ótimo local em uma vizinhança ainda pode ser melhorada em outra. Essa combinação funciona porque o GA descobre diferentes bacias de atração do espaço de busca, enquanto o VND refina cada solução dentro do escopo local de sua bacia. Quanto à capacidade dos quartos, a heurística combina as estratégias de penalização e de reparo descritas na Seção 2.5.

Quanto ao tomador de decisão, este trabalho adota uma postura *a priori*, por meio de um sistema de penalidades agregadas em uma única função de utilidade. No planejamento do censo, a equipe responsável precisa de uma alocação em tempo compatível com a rotina de planejamento, o que torna pouco prático gerar e analisar um conjunto de soluções de compromisso a cada novo planejamento. Os pesos das restrições e dos interesses dos *stakeholders* são, portanto, pré-parametrizados [2], permitindo que o algoritmo recomende uma

alocação de forma autônoma, sem intervenção do gestor durante a busca.

Em relação aos trabalhos revisados, este trabalho enfatiza três aspectos. O primeiro é a comparação direta entre a heurística e um *solver* comercial que resolve o mesmo modelo, sob o mesmo teto de tempo, em instâncias sintéticas de quatro portes, de 100 a 6400 pacientes, com dez conjuntos independentes por porte. O segundo é a garantia de viabilidade das soluções heurísticas por uma etapa de reparo da capacidade. O terceiro é o uso de um limite inferior combinatório, obtido diretamente dos dados, para avaliar a qualidade das soluções nas instâncias em que o *solver* não fornece um limite útil.

Com a base teórica acerca das características estruturais do problema estabelecida e a arquitetura híbrida (GA + VND) definida, o Capítulo 3 detalha a metodologia, apresentando o modelo matemático adotado e as estratégias de implementação das abordagens exata e heurística.

Capítulo 3

Metodologia

Este capítulo apresenta a metodologia utilizada para modelar e resolver o Problema de Alocação de Leitos (PBA). Primeiramente, são descritos o problema e as instâncias de teste. Em seguida, apresentam-se o modelo matemático, resolvido pela abordagem exata, e depois o Algoritmo Memético, base da abordagem heurística, com sua representação da solução, seus operadores e a etapa de reparo, e a sua implementação. Por fim, são descritas as métricas de avaliação e o protocolo experimental.

3.1 Descrição do problema

O Problema de Alocação de Leitos (PBA) abordado neste trabalho concentra-se no nível tático da gestão hospitalar, isto é, o planejamento do censo ao longo de um horizonte, no qual um conjunto conhecido de pacientes, com diferentes necessidades clínicas e datas de admissão e de alta definidas, deve ser alocado a um conjunto finito de quartos e leitos disponíveis nas diversas alas do hospital [8]. A alocação deve garantir a viabilidade clínica e, simultaneamente, maximizar o conforto do paciente e a eficiência dos recursos operacionais [2].

3.1.1 Parâmetros, variáveis e discretização do tempo

Para reduzir a complexidade computacional e refletir a rotina de censos hospitalares, o tempo é discretizado e modelado em dias consecutivos [6, 8]. O dia atua como a menor unidade indivisível de tempo do sistema. Dessa forma, as internações, altas e transferências ocorrem estritamente na transição de um dia para o outro [8].

O sistema recebe como entrada uma lista de pacientes que possuem uma data de admissão e uma data de alta, resultando em um tempo de permanência (LOS) ininterrupto [6]. Durante todo o LOS estipulado, o paciente deve obrigatoriamente permanecer alocado nas dependências do hospital, não sendo consideradas transferências para unidades externas.

O hospital é composto por quartos, cada um associado a uma especialidade clínica e com uma capacidade máxima definida (número de leitos). Como não há diferenciação de equipamentos vitais ou infraestrutura entre leitos de um mesmo quarto, os leitos tornam-se indistinguíveis. Conforme proposto por Ceschia e Schaerf [6], o problema pode ser simplificado passando de uma alocação baseada em leitos individuais para uma alocação direta em

quartos, o que reduz substancialmente o espaço de busca. Assim, as variáveis de decisão centrais definem em qual quarto o paciente passará cada dia do seu período de internação.

3.1.2 Restrições

O problema está sujeito a dois conjuntos de regras [6, 8]:

- **Restrições Rígidas (*Hard Constraints*):** Devem ser obrigatoriamente satisfeitas para que a solução seja viável:
  - *Garantia de Internação Contínua:* todo paciente admitido deve ocupar exatamente um quarto em cada dia de seu tempo de permanência.
  - *Capacidade do Quarto:* o número de pacientes alocados simultaneamente em um quarto não pode exceder a capacidade máxima (número de leitos) daquele quarto. Na abordagem heurística, essa restrição é relaxada e penalizada durante a busca local, e a viabilidade é restabelecida pela etapa de reparo (Subseção 3.3.6).
- **Restrições Flexíveis (*Soft Constraints*):** Representam preferências que, se violadas, geram penalidades na função objetivo, sem inviabilizar a solução:
  - *Especialidade:* alocar o paciente em um quarto da especialidade clínica adequada à sua patologia.
  - *Transferências:* evitar que o paciente mude de quarto durante a internação.
  - *Política de gênero:* evitar que pacientes de gêneros distintos ocupem, concomitantemente, o mesmo quarto.

Outras regras citadas na literatura, como agrupamento por faixa etária, requisitos de isolamento e a preferência do paciente por quarto individual ou enfermaria, não foram consideradas neste trabalho.

3.1.3 Pesos e abordagem multiobjetivo

A natureza multiobjetivo do PBA exige o balanceamento de interesses conflitantes, como entre minimizar transferências e manter cada paciente em um quarto de sua especialidade. Conforme discutido no Capítulo 2, este trabalho adota uma abordagem *a priori* [9]. Todos os objetivos e violações de restrições flexíveis são agregados em uma única função de utilidade através do Método das Somas Ponderadas [2, 8]. Dessa forma, pesos específicos (W) são atribuídos a cada violação, representando prioridades que, em uma aplicação real, seriam definidas pela administração hospitalar. Quanto maior o peso de um critério (como a penalidade por transferências), maior será o esforço do algoritmo para evitá-lo.

Os três critérios da função objetivo são comensuráveis, pois todos correspondem a contagens de eventos, sejam pacientes-dia em quarto de especialidade inadequada, transferências realizadas ou quartos-dia ocupados por pacientes de gêneros distintos. Por isso, não se aplica nenhuma normalização entre eles, e os pesos (Tabela 3.2) expressam diretamente as taxas de troca entre os critérios. Um paciente-dia em especialidade inadequada ($W_{spec} = 100$) equivale a duas transferências ($W_{transf} = 50$), ou seja, uma transferência vale meio paciente-dia

em especialidade inadequada. Um quarto-dia misto ($W_{gen} = 75$) equivale a 1,5 transferência, e um paciente-dia em especialidade inadequada equivale a cerca de 1,33 quarto-dia misto. Uma consequência direta dessas taxas é que, considerando apenas as parcelas de especialidade e de transferência, transferir um paciente de um quarto de especialidade inadequada para um quarto de sua especialidade compensa sempre que lhe resta ao menos um dia de internação, pois a transferência custa $W_{transf} = 50$ uma única vez, enquanto cada dia no quarto adequado economiza $W_{spec} = 100$. As transferências presentes nas soluções são, portanto, em parte uma escolha induzida pelos próprios pesos. Como Z é uma soma sobre pacientes-dia, quartos-dia e transferências, o seu valor cresce com o porte da instância e não é comparável entre portes. Por esse motivo, os resultados comparam as abordagens dentro de cada porte e, entre portes, utilizam métricas relativas, como porcentagens da carga de internação.

3.1.4 Dados e instâncias de teste

A literatura aponta a dificuldade de obter dados hospitalares reais e abertos para testar algoritmos de alocação, devido a questões de sigilo médico, e trabalhos como o de Demeester et al. [8] também recorrem a dados gerados aleatoriamente.

Dessa forma, as instâncias de teste foram geradas sinteticamente por um gerador próprio, descrito na Subseção 3.1.5. Os parâmetros do gerador, como a ocupação-alvo de 90% e o tempo médio de permanência de quatro dias, foram escolhidos para produzir cenários de alta ocupação, nos quais as decisões de alocação têm maior impacto.

Além disso, com o objetivo de isolar e avaliar o desempenho computacional e a qualidade da otimização, este trabalho utiliza dados determinísticos, sem fatores estocásticos como variações inesperadas no tempo de permanência ou cancelamentos repentinos, de modo que as diferenças entre as soluções se devem apenas aos métodos de otimização. Assim, os dias de admissão, as capacidades dos quartos e os tempos de internação são predefinidos e fornecidos ao modelo como parâmetros exatos, e cada instância é gerada a partir de uma semente fixa, o que torna os experimentos reprodutíveis.

3.1.5 Gerador de instâncias

As instâncias de teste são produzidas por um gerador sintético desenvolvido neste trabalho, em quatro portes (pequena, média, grande e muito grande), com 10, 40, 160 e 640 quartos, respectivamente. Todos os quartos têm capacidade de três leitos, e cada instância contém dez pacientes por quarto, totalizando de 100 a 6400 pacientes. Para cada porte são gerados dez conjuntos independentes, o que permite avaliar a variação dos resultados devida ao próprio sorteio dos dados, e não apenas à aleatoriedade da heurística.

Cada conjunto é gerado a partir de uma semente própria, calculada como $100b + k$, em que b é a semente base do porte (42, 43, 44 e 45, respectivamente) e $k \in \{1, \ldots, 10\}$ é o número do conjunto. Dessa forma, toda execução do gerador reproduz exatamente as mesmas instâncias, e nenhum par de conjuntos compartilha a mesma semente.

A especialidade de cada quarto e a especialidade requerida por cada paciente são sorteadas com a mesma distribuição, de 40% para Clínica Geral e 20% para cada uma das especialidades de Cirurgia, Cardiologia e Ortopedia. O gênero é sorteado com probabilidade igual entre feminino e masculino. O tempo de permanência é sorteado a partir de uma dis-

tribuição normal com média de quatro dias e desvio padrão de um dia, arredondado para o inteiro mais próximo e limitado inferiormente a um dia.

O dia de admissão de cada paciente não é sorteado, mas definido por uma estratégia de empacotamento, na qual os pacientes são ordenados do maior para o menor tempo de permanência, com desempate aleatório, e cada um é admitido no primeiro dia a partir do qual há leito livre no hospital durante toda a sua internação. O horizonte inicial é dimensionado para uma ocupação-alvo de 90% da capacidade total:

$$H = \left\lceil \frac{|P| \cdot \overline{LOS}}{0{,}9 \cdot \sum_{r \in R} c_r} \right\rceil + 2 \tag{3.1}$$

em que $\overline{LOS}$ é o tempo médio de permanência sorteado e, caso algum paciente não caiba no horizonte, ele é estendido. Definir a admissão por empacotamento, e não por sorteio, evita que a capacidade fique fragmentada ao longo do horizonte, o que reduziria a ocupação efetiva abaixo do alvo. Além disso, como o número de pacientes internados em cada dia nunca excede o total de leitos do hospital, toda instância gerada admite ao menos uma solução viável para o modelo da Seção 3.2.

A Tabela 3.1 resume as características das instâncias geradas, que dependem apenas dos dados e são iguais para qualquer solução de um mesmo conjunto. O horizonte de cerca de 15 dias e a ocupação média de 88% a 89% são consequências do tempo médio de permanência de quatro dias e da ocupação-alvo de 90%. O pico de 100% em todos os conjuntos decorre da admissão por empacotamento, em que os leitos ficam totalmente ocupados durante cerca de 12 dias, e a ocupação só cai nos últimos dias do horizonte, quando restam apenas as internações mais longas, o que explica o mínimo de 11% a 14%. Com o hospital lotado na maior parte do horizonte, não há folga de capacidade a ser explorada, cenário de alta ocupação associado, na literatura, à superlotação dos serviços de emergência [1, 3].

Tabela 3.1: Características das instâncias geradas (média $\pm$ desvio padrão entre os 10 conjuntos)

Característica	**Pequena**	**Média**	**Grande**	**Muito grande**
Horizonte (dias)	15,1 $\pm$ 0,6	15,1 $\pm$ 0,3	15,0 $\pm$ 0,0	15,0 $\pm$ 0,0
Dias com lotação total	12,4 $\pm$ 0,5	12,3 $\pm$ 0,5	12,1 $\pm$ 0,3	12,0 $\pm$ 0,0
Ocupação média (todo o horizonte)	87,9% $\pm$ 1,9%	89,0% $\pm$ 1,4%	89,2% $\pm$ 0,6%	88,9% $\pm$ 0,4%
Pico de ocupação diária	100,0% $\pm$ 0,0%	100,0% $\pm$ 0,0%	100,0% $\pm$ 0,0%	100,0% $\pm$ 0,0%
Mínimo de ocupação diária	11,3% $\pm$ 6,9%	14,3% $\pm$ 6,4%	12,3% $\pm$ 2,6%	11,0% $\pm$ 1,2%

3.2 Modelagem matemática

A formulação a seguir define o Problema de Alocação de Leitos (PBA) como um modelo de Programação Linear Inteira (PLI), resolvido diretamente pelo *solver* comercial Gurobi na abordagem exata.

A formulação adota a abordagem de linearização proposta por Ceschia e Schaerf [7] para rastrear as características dinâmicas de convivência nos quartos (como a política de separação de gêneros), introduzindo variáveis auxiliares que definem o estado de ocupação de cada quarto a cada dia.

3.2.1 Conjuntos e parâmetros

O modelo é fundamentado nos seguintes conjuntos, índices e parâmetros:

- P: Conjunto de todos os pacientes. Este conjunto é subdividido em pacientes do gênero feminino (P_F) e masculino (P_M), de modo que $P = P_F \cup P_M$.
- R: Conjunto de quartos disponíveis no hospital.
- D: Conjunto de dias do horizonte de planejamento (dia 1 ao último dia de alta entre todos os pacientes).
- $D_p = \{d_p^{\text{adm}}, \ldots, d_p^{\text{alta}}\} \subseteq D$: Subconjunto de dias contínuos em que o paciente p está internado, sendo d_p^{adm} o dia de admissão e d_p^{alta} o último dia de internação.
- $P_d = \{p \in P : d \in D_p\}$: Subconjunto dos pacientes internados no dia d.
- c_r: Capacidade máxima (número de leitos) do quarto r.
- $C_{p,r}$: Custo de alocação do paciente p ao quarto r. Este parâmetro penaliza alocações indesejadas, devido à inadequação da especialidade clínica do quarto ao diagnóstico do paciente [7].

$$C_{p,r} = \begin{cases} W_{spec}, & \text{se } e_p \neq e_r \\ 0, & \text{caso contrário} \end{cases} \tag{3.2}$$

  em que e_p é a especialidade clínica requerida pelo paciente p e e_r é a especialidade do quarto r.
- W_{spec}: Peso de penalidade por dia em que o paciente permanece em um quarto de especialidade inadequada.
- W_{transf}: Peso de penalidade atribuído a cada transferência de quarto sofrida pelo paciente durante a internação.
- W_{gen}: Peso de penalidade por violação da política de gênero (alocar pacientes de gêneros distintos no mesmo quarto no mesmo dia).

Os pesos adotados neste trabalho refletem a prioridade atribuída a cada um dos critérios considerados na função objetivo, sendo a adequação da especialidade o critério mais relevante, seguido da política de gênero e, por fim, do desconforto causado por uma transferência. Dessa forma, são adotados os valores apresentados na Tabela 3.2.

Tabela 3.2: Pesos adotados para os critérios da função objetivo

Pesos	**Valor**
W_{spec}	100
W_{gen}	75
W_{transf}	50

3.2.2 Variáveis de decisão

O modelo baseia-se em variáveis primárias para a alocação e variáveis auxiliares para a linearização dos custos [7]. As variáveis $x_{p,r,d}$ são definidas apenas para os dias de internação ($d \in D_p$), e as variáveis $t_{p,r,d}$, apenas para os dias de internação anteriores ao último ($d \in D_p \setminus \{d_p^{\text{alta}}\}$), pois não há transferência após a alta.

Variáveis principais (alocação e transferência):

- $x_{p,r,d} \in \{0,1\}$: vale 1 se o paciente p está alocado no quarto r no dia d.
- $t_{p,r,d} \in \{0,1\}$: vale 1 se o paciente p é transferido do quarto r do dia d para o dia $d+1$.

Variáveis de linearização (status do quarto): Para rastrear a mistura de gêneros de forma linear, definem-se as variáveis de status do quarto r no dia d:

- $f_{r,d} \in \{0,1\}$: vale 1 se há pelo menos uma paciente do gênero feminino no quarto r no dia d.
- $m_{r,d} \in \{0,1\}$: vale 1 se há pelo menos um paciente do gênero masculino no quarto r no dia d.
- $b_{r,d} \in \{0,1\}$: vale 1 se o quarto r no dia d possui pacientes de ambos os gêneros.

3.2.3 Função objetivo

A função objetivo Z visa minimizar a soma total das penalidades. Ela é composta pelos custos clínicos da alocação ($C_{p,r}$), pelas penalidades aplicadas a cada transferência de quarto realizada (W_{transf}) e pelas penalidades geradas pelo uso de quartos mistos (W_{gen}):

$$\begin{aligned}\min Z = &\sum_{p\in P}\sum_{r\in R}\sum_{d\in D_p} C_{p,r}\cdot x_{p,r,d} + \sum_{p\in P}\sum_{r\in R}\sum_{d\in D_p\setminus\{d_p^{\text{alta}}\}} W_{transf}\cdot t_{p,r,d} \\ &+ \sum_{r\in R}\sum_{d\in D} W_{gen}\cdot b_{r,d}\end{aligned} \tag{3.3}$$

3.2.4 Restrições

O modelo está sujeito às seguintes restrições:

1. Atribuição Única e Contínua: Cada paciente p deve estar alocado a exatamente um quarto durante todos os dias contínuos da sua internação (D_p).

$$\sum_{r\in R} x_{p,r,d} = 1, \quad \forall p \in P, \forall d \in D_p \tag{3.4}$$

2. Limite de Capacidade do Quarto: O número total de pacientes acomodados simultaneamente em um quarto r no dia d não pode exceder sua capacidade máxima c_r.

$$\sum_{p\in P_d} x_{p,r,d} \le c_r, \quad \forall r \in R, \forall d \in D \tag{3.5}$$

3. Rastreamento de Transferências: A variável de transferência deve ser ativada caso o paciente ocupe o quarto r no dia d, mas não esteja mais alocado nele no dia subsequente $d+1$ da sua internação [7].

$$t_{p,r,d} \geq x_{p,r,d} - x_{p,r,d+1}, \quad \forall p \in P, \forall r \in R, \forall d \in D_p \setminus \{d_p^{\text{alta}}\} \tag{3.6}$$

4. Identificação de Presença Feminina: Se uma paciente do conjunto feminino (P_F) for alocada ao quarto r no dia d, a *flag* indicadora de presença feminina desse quarto deve ser ativada.

$$f_{r,d} \geq x_{p,r,d}, \quad \forall p \in P_F, \forall r \in R, \forall d \in D_p \tag{3.7}$$

5. Identificação de Presença Masculina: Semelhantemente à restrição anterior, a *flag* deve ser ativada para pacientes do conjunto masculino (P_M).

$$m_{r,d} \geq x_{p,r,d}, \quad \forall p \in P_M, \forall r \in R, \forall d \in D_p \tag{3.8}$$

6. Detecção de Mistura de Gêneros (Quarto Misto): Esta restrição força a variável $b_{r,d}$ a assumir o valor 1 caso o quarto possua representantes de ambos os gêneros no mesmo dia, acionando a penalidade W_{gen} na função objetivo [7].

$$b_{r,d} \geq m_{r,d} + f_{r,d} - 1, \quad \forall r \in R, \forall d \in D \tag{3.9}$$

3.2.5 Domínio das variáveis

Conforme definido anteriormente, o domínio de todas as variáveis restringe-se a valores binários:

$$x_{p,r,d}, t_{p,r,d}, f_{r,d}, m_{r,d}, b_{r,d} \in \{0, 1\} \tag{3.10}$$

3.3 Algoritmo Memético

O Algoritmo 1 resume o fluxo do Algoritmo Memético adotado na abordagem heurística, cujos componentes são detalhados nas subseções seguintes. O símbolo F denota o *fitness*, conforme a Equação (3.12), e g_{est} conta as gerações consecutivas sem melhora da melhor solução.

Para simplificar, o Algoritmo 1 omite o controle de tempo dos reparos. Conforme a Subseção 3.3.6, o VND é encerrado antes do fim do teto, com antecedência suficiente para o último reparo, e uma nova geração só é criada se o reparo de seus filhos ainda couber no tempo restante. Caso contrário, o tempo é usado pelo VND na população atual, que já é viável.

3.3.1 Representação da solução

Um dos passos mais críticos no *design* de qualquer meta-heurística, especialmente em Algoritmos Genéticos (GA), é a escolha de uma representação de solução (cromossomo) que seja ao mesmo tempo eficiente em termos de memória e fácil de ser manipulada pelos operadores de busca [9].

Para o Problema de Alocação de Leitos (PBA), a representação computacional de um indivíduo na população precisa espelhar a variável de decisão $x_{p,r,d}$. Em vez de utilizar

Algoritmo 1 Algoritmo Memético com etapa de reparo da capacidade

Entrada: pacientes P, quartos R, teto de tempo T
Saída: melhor solução encontrada s^*, sem violação de capacidade

```
Pop ← 30 indivíduos, 20% gulosos e 80% aleatórios                ▷ Subseção 3.3.2
REPARAR cada indivíduo de Pop                                    ▷ Subseção 3.3.6
ordenar Pop por F crescente
s* ← primeiro indivíduo de Pop
g_est ← 0
enquanto o teto de tempo T não for atingido faça
    Filhos ← cópias dos 3 indivíduos de elite de Pop
    enquanto |Filhos| < 30 faça
        p1, p2 ← dois pais escolhidos por torneio de tamanho 3 em Pop
        c ← CRUZAMENTO(p1, p2)                                   ▷ Subseção 3.3.5
        se g_est ≥ 5 então
            mutar exatamente 35% dos pacientes de c
        senão
            mutar cada paciente de c com probabilidade de 5%
        fim se
        Filhos ← Filhos ∪ {c}
    fim enquanto
    substituir 3 filhos não elite por novos indivíduos aleatórios (imigrantes)
    REPARAR cada filho não elite
    aplicar o VND à elite e a 9 filhos não elite sorteados       ▷ Subseção 3.3.4
    REPARAR cada indivíduo de Filhos
    Pop ← Filhos, ordenados por F crescente
    se F(primeiro indivíduo de Pop) < F(s*) então
        s* ← primeiro indivíduo de Pop
        g_est ← 0
    senão
        g_est ← g_est + 1
    fim se
fim enquanto
retorne s*
```

uma matriz tridimensional esparsa, que consumiria processamento excessivo, a solução é representada como um *array* (ou dicionário de dados), em que cada índice representa um paciente $p \in P$. O valor (alelo) contido nesse índice é um subvetor que lista o quarto $r \in R$ ocupado pelo paciente em cada dia contínuo $d \in D_p$ do seu tempo de permanência (LOS) [8, 6].

3.3.2 Solução inicial

Em algoritmos genéticos e de busca local, a qualidade da população inicial impacta diretamente a velocidade de convergência. Neste trabalho, a população inicial combina dois construtores com vieses distintos. A maior parte dos indivíduos (80%) é gerada de forma semigulosa e aleatória [6], em que, para cada paciente, sorteia-se um quarto da especiali-

dade que ele requer, mantendo-o no mesmo quarto do primeiro ao último dia de internação ($t_{p,r,d} = 0$), sem considerar a capacidade.

Os 20% restantes são construídos de forma gulosa e ciente da capacidade. Os pacientes são processados em ordem aleatória, e cada um é alocado a um quarto de sua especialidade com leito livre em todos os dias da internação ou, não havendo, a qualquer quarto com leito livre e, só em último caso, a um quarto sem vaga. Essa proporção foi definida experimentalmente (Subseção 3.6.1). Construir toda a população de forma gulosa produziu resultados piores, pois uma população gerada por um único construtor fica estruturalmente homogênea e oferece pouca variedade ao cruzamento.

3.3.3 Violação de capacidade e *fitness*

Na abordagem heurística, diferentemente da abordagem exata, a capacidade dos quartos é tratada como uma restrição flexível durante a busca local. Conforme discutido na Seção 2.5, rejeitar toda solução que excedesse a capacidade criaria descontinuidades no espaço de busca e restringiria excessivamente os operadores. Em vez disso, o excesso de ocupação é contabilizado pela violação de capacidade V_{cap}, que soma, para cada quarto e cada dia, quantos pacientes excedem o número de leitos disponíveis:

$$V_{cap} = \sum_{r \in R} \sum_{d \in D} \max\left(0,\ \sum_{p \in P_d} x_{p,r,d} - c_r\right) \tag{3.11}$$

em que $\sum_{p \in P_d} x_{p,r,d}$ é o número de pacientes alocados no quarto r no dia d, e o termo $\max(0, \cdot)$ considera apenas a parcela que ultrapassa a capacidade c_r, sendo nulo quando o quarto está dentro do limite. Cada unidade de V_{cap} corresponde, portanto, a um paciente-dia alocado além do número de leitos do quarto, sendo uma solução viável se, e somente se, $V_{cap} = 0$.

Os indivíduos são então comparados pelo *fitness* F, que acrescenta à função objetivo a violação de capacidade ponderada:

$$F = Z + W_{cap} \cdot V_{cap} \tag{3.12}$$

em que Z é o valor da função objetivo, conforme a Equação (3.3), e W_{cap} é o peso de penalidade atribuído a cada paciente-dia alocado além da capacidade.

Como W_{cap} não faz parte da função objetivo (Z), o valor comparado ao Gurobi é sempre Z e a violação de capacidade V_{cap} é reportada separadamente. Adotou-se $W_{cap} = 1900$, valor obtido por uma varredura experimental (Subseção 3.6.1). A penalidade, sozinha, não garante soluções viáveis, e por isso o algoritmo inclui uma etapa de reparo da capacidade (Subseção 3.3.6), que elimina a violação dos indivíduos a cada geração.

3.3.4 Estruturas de vizinhança e busca local

Na arquitetura do Algoritmo Memético adotada, a busca local (VND) é aplicada a cada geração, após o cruzamento e a mutação. Como uma solução que é ótimo local em uma vizinhança ainda pode ser melhorada em outra, o VND alterna sistematicamente entre diferentes estruturas de vizinhança [9].

Baseando-se nas abordagens de Ceschia e Schaerf [7, 6] e Demeester et al. [8] para a alocação de pacientes a quartos, as seguintes estruturas de vizinhança formam o núcleo do otimizador local:

- N_1**: Mudança de Quarto (*Change Room - CR*):** Este movimento consiste em transferir virtualmente um paciente p de seu quarto atual para um novo quarto r' durante toda a sua estadia. Trata-se de uma exploração primária do espaço de busca, mantendo o número de transferências intra-hospitalares igual a zero, mas buscando quartos que reduzam o custo estático $C_{p,r}$ [6, 8].

- N_2**: Troca entre Pacientes (*Swap Patients - SP*):** Consiste em permutar os quartos atribuídos a dois pacientes distintos p_1 e p_2, cujos dias de internação possuam alguma sobreposição. Esse movimento é vital, pois, frequentemente, a mudança de um paciente isolado (N_1) é inviabilizada por limites de capacidade. A troca simultânea permite que o algoritmo contorne a falta de leitos livres e acomode melhor os pacientes sem inflar a ocupação máxima do quarto [6, 8].

- N_3**: Mudança Parcial de Quarto (*Partial Change Room - PCR*):** As vizinhanças anteriores exploram apenas soluções sem transferências. No entanto, quartos ideais podem estar disponíveis apenas em parte do período de internação. O movimento PCR atua no espaço de soluções expandido, com transferências, ao selecionar um dia d no meio da internação de um paciente p e alterar seu quarto apenas daquele dia em diante. Embora esse movimento ative a variável de transferência $t_{p,r,d}$ e adicione a penalidade de transferência (W_{transf}) à função objetivo, ele confere a flexibilidade necessária para resolver gargalos de capacidade em dias de pico [6].

- N_4**: Troca Parcial entre Pacientes (*Partial Swap Patients - PSP*):** Generalização de N_2 proposta por Ceschia e Schaerf [7]. Em vez de permutar as internações inteiras de dois pacientes, troca os quartos apenas nos dias em que as internações se sobrepõem, preservando o restante da sequência de cada um. Com isso, resolve conflitos restritos a parte da internação sem desfazer transferências já construídas por N_3.

A busca local (VND) mantém uma fila de pacientes ativos, que contém inicialmente todos os pacientes, em ordem aleatória. Para cada paciente retirado da fila, tenta as vizinhanças na ordem $N_1 \rightarrow N_2 \rightarrow N_3 \rightarrow N_4$ e aplica o primeiro movimento que melhore o *fitness*. A busca termina quando a fila se esvazia, isto é, quando nenhum paciente admite movimento de melhora, o que caracteriza um ótimo local em relação às quatro vizinhanças, ou quando se esgota o limite de avaliações ou o tempo disponível. Para reduzir o custo desse processo, adotam-se três mecanismos:

1. *Don't-look bits*, em que um paciente sem movimento de melhora sai da fila e só volta a ela se um movimento posterior alterar sua alocação.

2. Amostragem de candidatos, em que cada movimento de mudança de quarto (N_1 e N_3) avalia no máximo 10 quartos de destino, escolhidos, entre 30 quartos sorteados, como os que têm menos dias lotados no período em questão, e cada movimento de troca (N_2 e N_4) avalia no máximo 10 pacientes, sorteados entre os que têm internação sobreposta à do paciente.

3. Limite de avaliações por chamada do VND igual a 250 vezes o número de pacientes ($250 \cdot |P|$), que impede que o refino de um único indivíduo consuma uma geração inteira nas instâncias grandes.

3.3.5 Operadores do Algoritmo Genético

A população tem tamanho fixo de 30 indivíduos em todos os portes de instância. A cada geração, o algoritmo executa os passos a seguir, até que o teto de tempo seja atingido:

1. **Elitismo**: os 10% melhores indivíduos da geração anterior são copiados para a nova geração.

2. **Reprodução**: o restante da população é preenchido por filhos gerados a partir de dois pais escolhidos por torneio de tamanho 3, aos quais se aplicam cruzamento e mutação.

3. **Imigrantes**: três filhos não elite, o equivalente a 10% da população, são substituídos por indivíduos novos, gerados pelo construtor aleatório da solução inicial.

4. **Reparo dos filhos**: os filhos não elite e os imigrantes passam pela etapa de reparo (Subseção 3.3.6), de modo que chegam à busca local sem violação de capacidade.

5. **Busca local**: o VND é aplicado a todos os indivíduos de elite e a nove dos demais, o equivalente a 30% da população, sorteados. Os indivíduos de elite são incluídos porque, como não passam por cruzamento nem mutação, permaneceriam inalterados enquanto mantivessem sua posição na população, escolha validada experimentalmente (Subseção 3.6.1).

6. **Reparo após a busca local**: como o VND é guiado pelo *fitness*, ele pode aceitar um movimento que exceda a capacidade se a redução de Z compensar a penalidade, e por isso os indivíduos refinados são reparados novamente.

7. **Atualização**: a população é ordenada pelo *fitness*, e é registrada a melhor solução encontrada até então.

A população inicial também passa pelo reparo antes da primeira geração. Assim, a população que chega à seleção por torneio é sempre composta apenas de soluções viáveis, e a capacidade permanece flexível somente dentro da busca local.

Os operadores de cruzamento e de mutação usados no passo de reprodução foram adaptados à estrutura do problema e são descritos a seguir.

- **Cruzamento por blocos de quarto**: Com probabilidade de 0,9, o filho é gerado por cruzamento e, caso contrário, é uma cópia de um dos pais. No cruzamento, um dos pais é sorteado como primário, e os pacientes são agrupados de acordo com o quarto que ocupam no primeiro dia de internação nesse pai. Cada grupo é herdado em bloco do pai primário com probabilidade de 0,5, e os pacientes dos demais grupos recebem a sequência de quartos que tinham no outro pai. Herdar o grupo inteiro preserva as combinações de colegas de quarto já ajustadas, especialmente quanto ao gênero, que um cruzamento paciente a paciente tenderia a desfazer.

- **Mutação**: Cada paciente do filho é mutado com probabilidade de 5%, por um movimento sorteado entre três tipos análogos às vizinhanças do VND, que são a mudança do quarto em toda a internação (N_1) e a mudança a partir de um dia sorteado (N_3), ambas para um quarto sorteado da especialidade do paciente, e a troca de quartos com outro paciente nos dias em que ambos estão internados (N_4). Ao contrário da busca local, a mutação não verifica se o movimento melhora a solução. Essa intensidade é ajustada conforme o progresso da busca. Quando a melhor solução não melhora por 5 gerações consecutivas, a mutação passa a alterar exatamente 35% dos pacientes de cada filho, voltando aos 5% assim que uma nova melhor solução é encontrada. Como o algoritmo só termina pelo teto de tempo, esse reforço pode ser acionado várias vezes em uma mesma execução.

Em conjunto, esses mecanismos equilibram as duas forças da busca. O elitismo e o VND intensificam a busca em torno das melhores soluções, enquanto a mutação, seu reforço por estagnação e os imigrantes mantêm a diversidade da população. Os parâmetros adotados estão resumidos na Tabela 3.3.

Tabela 3.3: Parâmetros do Algoritmo Memético

Parâmetro	**Valor**
Tamanho da população	30
População inicial gulosa / aleatória	20% / 80%
Tamanho do torneio	3 indivíduos
Taxa de cruzamento	0,9
Probabilidade de herança de um bloco	0,5
Taxa de mutação	5% dos pacientes
Mutação reforçada	35% dos pacientes, após 5 gerações sem melhora
Elite	10% da população
Imigrantes	10% da população (3 indivíduos)
Busca local	Elite + 30% da população (9 indivíduos)
Candidatos por movimento do VND	10 quartos, escolhidos entre 30 sorteados
Limite de avaliações do VND	$250 \cdot \lvert P \rvert$ por chamada
W_{cap}	1900
Reparo da capacidade	Antes e depois do VND, a cada geração
Candidatos por realocação do reparo	10 quartos com leito livre

3.3.6 Etapa de reparo da capacidade

Uma solução viável sempre pode ser obtida a partir de uma solução que viola a capacidade. Como o gerador garante que, em todos os dias, o número de pacientes internados não excede o total de leitos do hospital (Subseção 3.1.5), sempre que um quarto está acima da capacidade existe outro quarto com leito livre no mesmo dia. Cada paciente-dia excedente pode, então, ser realocado para um quarto com vaga apenas naquele dia, o que elimina uma unidade de V_{cap} sem criar violação em outro quarto. No pior caso, essa realocação acrescenta a Z um paciente-dia em especialidade inadequada (W_{spec}), um quarto-dia misto (W_{gen}) e duas transferências, uma para o novo quarto e outra de volta ao original ($2W_{transf}$), isto é,

$$\Delta Z \leq W_{spec} + W_{gen} + 2W_{transf} = 275 \tag{3.13}$$

por unidade de V_{cap} eliminada. Como $W_{cap} = 1900$ é maior que esse custo, cada realocação também reduz o *fitness*.

A etapa de reparo aplica essa ideia escolhendo, a cada passo, a realocação mais barata. Para cada quarto-dia acima da capacidade, são avaliados todos os seus ocupantes e até 10 quartos com leito livre naquele dia, priorizando os da especialidade requerida pelo paciente. Para cada par de paciente e quarto de destino, avaliam-se dois movimentos, que são mover apenas aquele dia e mover o maior bloco contínuo da internação, em torno daquele dia, passado no mesmo quarto e com leito livre no destino, o que evita a transferência de volta ao quarto original. Aplica-se o movimento de menor variação de *fitness*, isto é, o que mais reduz o *fitness*, e o processo se repete até que $V_{cap} = 0$. Como cada movimento reduz V_{cap} em ao menos uma unidade e nunca cria violação em outro quarto, o reparo sempre termina com uma solução viável e, como a realocação de um único dia está entre as opções avaliadas, o *fitness* da solução reparada é sempre menor que o da original, conforme a Equação (3.13).

O tempo gasto no reparo é contabilizado dentro do teto de tempo da heurística. Para que os reparos da última geração também terminem dentro do teto, a busca local é encerrada antes do fim do teto, com uma antecedência igual ao maior tempo de reparo após a busca local observado até então, e, quando o tempo restante não comporta o reparo dos filhos de uma nova geração, ele é usado pelo VND na população atual, que já é viável.

3.4 Implementação

3.4.1 Avaliação procedural e simplificação do modelo

Embora a Seção 3.2 apresente a formulação matemática exata e completa do problema por meio de Programação Linear Inteira (PLI), necessária para a execução com *solvers* comerciais como o Gurobi, a implementação computacional da meta-heurística pode adotar uma abordagem mais enxuta para a avaliação da função objetivo [8].

Em modelos estritamente lineares, a detecção de penalidades dinâmicas, como a violação da política de gênero no mesmo quarto, exige a criação de múltiplas variáveis de decisão auxiliares (por exemplo, $f_{r,d}$, $m_{r,d}$ e $b_{r,d}$) e restrições de amarração para cada quarto e cada dia do horizonte de planejamento [7]. Como o número de variáveis cresce com o produto entre o número de pacientes-dia e o número de quartos, um hospital de grande porte resulta em um modelo muito grande, com cerca de 28,6 milhões de variáveis na instância muito grande deste trabalho, o que torna o modelo intratável em tempo hábil [8].

Na arquitetura do Algoritmo Memético implementada neste trabalho (em Python), contorna-se essa limitação através da **avaliação procedural**. Como a meta-heurística explora o espaço de busca gerando arranjos completos (ou vizinhanças) a cada iteração, o algoritmo possui conhecimento total e imediato sobre o estado de ocupação de qualquer quarto r no dia d.

Dessa forma, a heurística dispensa a declaração das variáveis auxiliares de linearização. O custo é mantido de forma incremental. A solução guarda, para cada quarto e dia, o conjunto de ocupantes e a contagem por gênero, e cada movimento atualiza apenas os quartos e dias que foram alterados. Assim, avaliar um movimento custa proporcionalmente ao tempo de permanência do paciente movido, e não ao tamanho da instância.

Essa diferença entre a abordagem exata, que representa o modelo por restrições algébricas, e a abordagem heurística, que o avalia por meio de estruturas de dados, permite ao Algoritmo Memético avaliar entre cerca de 43 e 55 mil soluções candidatas por segundo. A

taxa diminui apenas moderadamente com o porte, de aproximadamente 55 mil na instância pequena para 43 mil na muito grande, embora esta tenha 64 vezes mais pacientes.

3.4.2 Ferramentas e ambiente de execução

O algoritmo heurístico proposto, bem como a modelagem de dados e rotinas de avaliação, foram desenvolvidos em Python 3.13, juntamente com as bibliotecas NumPy, Pandas e Matplotlib, com o *solver* exato executado com o Gurobi 13.0.1.

Todos os experimentos foram executados em um computador Apple MacBook Air com processador Apple M4, de 10 núcleos (4 de desempenho e 6 de eficiência), 16 GB de memória unificada e sistema operacional macOS 26.6.2, com Python 3.13.7. A heurística é executada em um único núcleo, enquanto o Gurobi utiliza, por padrão, todos os núcleos disponíveis. Para reduzir o tempo total de execução, o Gurobi foi executado em dois processos simultâneos na mesma máquina, e a heurística, em quatro processos simultâneos, um por núcleo de desempenho, de modo que parte da capacidade de processamento foi compartilhada entre eles.

Para o controle de versionamento e armazenamento do código-fonte, utiliza-se a ferramenta Git, com o projeto hospedado publicamente na plataforma GitHub, onde o repositório oficial do trabalho pode ser acessado [12].

O ambiente de desenvolvimento foi estruturado seguindo boas práticas de engenharia de *software* para garantir a reprodutibilidade dos experimentos. A execução do projeto é isolada através de um ambiente virtual (`venv`), com todas as dependências de bibliotecas e suas respectivas versões documentadas no arquivo `requirements.txt`. A arquitetura do código está modularizada no diretório `src`, no qual o *script* `main.py` atua como o orquestrador principal da otimização, interligando a fonte de dados, a avaliação da função objetivo e os operadores do Algoritmo Memético (GA + VND).

3.5 Métricas para avaliação de desempenho

A avaliação de desempenho de algoritmos heurísticos exige um planejamento experimental rigoroso para comprovar sua eficácia e eficiência frente ao problema proposto. Conforme definido por Talbi [9], para a análise de algoritmos de otimização, o desempenho do Algoritmo Memético desenvolvido é medido sob três perspectivas principais, que são a qualidade da solução, a eficiência computacional e a robustez estatística.

- **Eficácia (Qualidade da Solução):** A qualidade das alocações geradas é avaliada primariamente pelo valor final alcançado na função objetivo Z, conforme a Equação (3.3). Como o PBA é um problema de minimização, valores menores indicam um melhor balanceamento das restrições flexíveis. Adicionalmente, para cada solução são registrados o valor de cada parcela de Z e métricas operacionais explícitas, como o número de pacientes-dia em quarto de especialidade inadequada, o número de transferências, o número de quartos-dia mistos, a violação de capacidade (V_{cap}), a taxa de ocupação média e por especialidade e o pico de ocupação diária do hospital. Para que as duas abordagens sejam avaliadas de forma idêntica, Z, suas parcelas e todas as métricas operacionais são recalculados a partir das variáveis de alocação $x_{p,r,d}$ de cada solução, pelo mesmo avaliador utilizado na heurística, e não a partir das variáveis

auxiliares do modelo exato. Isso é necessário porque as variáveis $t_{p,r,d}$, $f_{r,d}$, $m_{r,d}$ e $b_{r,d}$ aparecem apenas em restrições de desigualdade, conforme as Equações (3.6) a (3.9), e poderiam valer 1 sem que a transferência ou o quarto misto correspondente ocorresse. O que as mantém em zero é a minimização da função objetivo, garantida apenas no ótimo. Nas 40 execuções do Gurobi, o valor de Z recalculado coincidiu com o valor da função objetivo reportado pelo *solver*, o que indica que essas variáveis não estavam infladas nas soluções obtidas.

Como a abordagem heurística trata a capacidade como restrição flexível durante a busca local, a viabilidade das soluções reportadas é verificada à parte. Para que a violação, quando houver, possa ser comparada entre portes, o valor absoluto de V_{cap}, conforme a Equação (3.11), não basta, pois tende a crescer com o porte da instância. Por esse motivo, V_{cap} é comparada à carga de internação N_{pd}, isto é, ao número total de pacientes-dia da instância:

$$N_{pd} = \sum_{p \in P} |D_p| \tag{3.14}$$

em que $|D_p|$ é o número de dias de internação do paciente p. Como cada unidade de V_{cap} corresponde a um paciente-dia, a violação nunca excede a carga de internação ($V_{cap} \leq N_{pd}$), que funciona, assim, como seu limite superior. A razão entre as duas grandezas define a taxa de violação de capacidade ρ_{cap}:

$$\rho_{cap} = \frac{V_{cap}}{N_{pd}} \times 100\% \tag{3.15}$$

que expressa a porcentagem da demanda de internação que ficou sem leito. Assim, $\rho_{cap} = 0\%$ indica uma solução viável, e valores próximos de zero indicam soluções próximas da viabilidade.

Para atestar a eficácia do Algoritmo Memético, as soluções heurísticas são comparadas às obtidas pelo método exato, com o *solver* Gurobi, executado sobre o modelo completo da Seção 3.2 com o mesmo teto de tempo concedido a cada execução da heurística. Ao final do tempo, o Gurobi fornece a melhor solução inteira encontrada (limite superior), o melhor limite inferior (LB, do inglês *lower bound*) do *branch-and-bound* e o *gap* de otimalidade entre eles. Quando o *gap* é nulo, a solução é ótima comprovada. Caso contrário, o limite inferior indica o quanto, no máximo, qualquer solução poderia melhorar.

Não foi construído um modelo separado para a relaxação linear. Em testes preliminares, essa relaxação isolada não foi resolvida em 300 segundos na instância grande (1600 pacientes) e, na muito grande (6400 pacientes), apenas a montagem do modelo, com cerca de 28,6 milhões de variáveis, consumiu cerca de 289 segundos. Utilizou-se, em vez disso, o limite inferior reportado pelo próprio *solver* ao fim do tempo, que corresponde à relaxação da raiz fortalecida por *presolve* e planos de corte e, quando o *solver* avança na árvore de busca, aprimorada pela exploração dos nós do *branch-and-bound*.

Esse limite só deixa de ser trivial depois que a relaxação da raiz é concluída. Antes disso, o *solver* reporta zero, valor que vale para qualquer solução, pois todos os coeficientes da função objetivo são não negativos. Nesses casos, o *gap* do *solver* não é reportado, o que é coerente com a limitação dos métodos exatos discutida na Subseção 2.2.1 e na Seção 3.4.

Para que a distância das soluções ao ótimo possa ser avaliada mesmo quando o *solver* não fornece um limite útil, utiliza-se também um limite inferior combinatório, calculado diretamente a partir dos dados de cada conjunto, sem recorrer ao *solver*. Ele segue a ideia de relaxação utilizada por Ceschia e Schaerf [7] para obter limites inferiores para o PAS e decorre da restrição de capacidade. Se, em um dia, há mais pacientes internados que requerem determinada especialidade do que leitos em quartos dessa especialidade, os pacientes excedentes necessariamente ocupam quartos de outra especialidade naquele dia, em qualquer solução viável. Por exemplo, se em um dia estão internados 20 pacientes que requerem Cirurgia e os quartos de Cirurgia somam 15 leitos, ao menos 5 desses pacientes estão em quartos de outra especialidade nesse dia, qualquer que seja a alocação.

Formalmente, seja S o conjunto de especialidades, $n_{s,d} = |\{p \in P : e_p = s,\ d \in D_p\}|$ o número de pacientes internados no dia d que requerem a especialidade s, e $c_s = \sum_{r \in R : e_r = s} c_r$ o total de leitos dos quartos da especialidade s. O número mínimo de pacientes-dia em quarto de especialidade inadequada, em qualquer solução viável, é

$$M = \sum_{d \in D} \sum_{s \in S} \max\left(0,\ n_{s,d} - c_s\right) \tag{3.16}$$

Como cada paciente-dia em especialidade inadequada custa W_{spec} na função objetivo, conforme a Equação (3.3), e as parcelas de transferência e de quartos mistos são não negativas, o valor ótimo Z^* satisfaz

$$Z^* \geq LB_{comb} = W_{spec} \cdot M \tag{3.17}$$

Esse limite considera apenas a parcela de especialidade e ignora as transferências e os quartos mistos, de modo que tende a ser mais fraco que o do *solver* quando este conclui a relaxação da raiz. Em compensação, é obtido em tempo desprezível para qualquer porte de instância. Em cada conjunto, adota-se como limite inferior L o maior entre o limite reportado pelo *solver* e LB_{comb}.

A partir de L, avalia-se o quanto uma solução de valor Z pode estar acima do ótimo. Como $L \leq Z^* \leq Z$,

$$\frac{Z - Z^*}{Z^*} \leq \frac{Z - L}{L} \tag{3.18}$$

isto é, a razão $(Z - L)/L$, denominada excesso máximo sobre o ótimo, indica o quanto, no máximo, a solução está acima do ótimo. Diferentemente do *gap* do *solver*, calculado em relação à própria solução, $(Z - LB)/Z$, essa razão tem L no denominador, pois mede o excesso em relação ao ótimo.

Para os quartos mistos, utiliza-se um mínimo análogo, mais simples, que decorre da paridade. Nos dias de lotação total, em que o número de pacientes internados é igual ao total de leitos, todos os quartos estão com os três leitos ocupados em qualquer

solução viável. Se, em um desses dias, o número de pacientes do gênero feminino não for múltiplo de três, não é possível distribuí-las apenas em quartos completos de um único gênero, e ao menos um quarto estará misto. Sendo n_d o número de pacientes internados no dia d e n_d^F o número de pacientes do gênero feminino entre eles, o número de quartos-dia mistos de qualquer solução viável é ao menos

$$G = \left| \left\{ d \in D : n_d = \sum_{r \in R} c_r \text{ e } n_d^F \bmod 3 \neq 0 \right\} \right| \tag{3.19}$$

Esse mínimo considera apenas os dias de lotação total e ignora a interação com a especialidade dos quartos, sendo, por isso, fraco, mas permite avaliar a heurística também nesse critério.

- **Eficiência Computacional:** Ambos os métodos recebem o mesmo teto de tempo por execução, que cresce com o tamanho da instância (Seção 3.6). A heurística não possui critério de parada por estagnação e sempre utiliza o teto completo, de modo que o tempo de execução, por si só, não diferencia os métodos. Por isso, o esforço computacional é medido pelo número de avaliações da função objetivo dentro do teto. No Gurobi, esse número corresponde às iterações do simplex e, na heurística, à soma das avaliações feitas pelo GA, pela busca local e pelo reparo. As duas medidas não têm a mesma unidade, pois uma iteração do simplex não corresponde à avaliação de uma solução completa, e por isso são comparadas pela forma como variam com o porte, e não pelo valor absoluto. A velocidade de convergência é acompanhada pela curva do melhor *fitness* a cada geração e pela comparação com o custo da população inicial. Os tetos adotados, de 30 a 1920 segundos, foram definidos para comparar os métodos em condições iguais e não representam o tempo de resposta esperado em operação.

- **Análise Estatística e Robustez:** Devido à natureza estocástica dos Algoritmos Genéticos e à aleatoriedade na exploração de vizinhanças do VND, uma única execução não é suficiente para atestar a qualidade do método [9]. Portanto, cada conjunto de dados é resolvido por 10 execuções independentes da heurística, cada uma com sua própria sequência aleatória. Para cada conjunto, reportam-se o melhor valor, a média e o coeficiente de variação dessas execuções, sendo a dispersão visualizada em gráficos do tipo *boxplot*. Além disso, cada porte de instância é composto por 10 conjuntos gerados independentemente, o que permite separar duas fontes de variação. A primeira é a variação entre execuções, devida à aleatoriedade da heurística, e a segunda é a variação entre conjuntos, devida ao sorteio da instância.

3.6 Protocolo experimental

Para cada porte de instância, cada um dos dez conjuntos é resolvido uma vez pela abordagem exata e várias vezes pela abordagem heurística, com o mesmo teto de tempo por execução para as duas. A Tabela 3.4 resume a configuração adotada.

Tabela 3.4: Configuração experimental por porte de instância

Porte	Quartos	Pacientes	Teto por execução	Execuções da heurística	Conjuntos
Pequena	10	100	30 s	10	10
Média	40	400	120 s	10	10
Grande	160	1600	480 s	10	10
Muito grande	640	6400	1920 s	10	10

O teto de tempo cresce com o porte da instância na mesma proporção que o número de pacientes. Como cada porte tem quatro vezes mais pacientes que o anterior, o teto é multiplicado por quatro a cada porte, partindo de 30 segundos na instância pequena, o que mantém constante o tempo disponível por paciente, de 0,3 segundo. Todos os portes utilizam o mesmo número de execuções da heurística, 10 por conjunto.

As instâncias têm semente fixa (Subseção 3.1.5), mas cada execução da heurística utiliza uma sequência aleatória própria, de modo que as execuções de um mesmo conjunto constituem amostras independentes. Na heurística, o teto inclui o tempo gasto na etapa de reparo (Subseção 3.3.6). Na abordagem exata, o teto de tempo se aplica à otimização, mas, a fim de possibilitar a geração de algum resultado, o tempo de construção do modelo no Gurobi não é contabilizado. Na instância muito grande, por exemplo, essa construção leva quase cinco minutos.

3.6.1 Calibração de parâmetros

Os parâmetros do Algoritmo Memético foram calibrados por meio de testes A/B isolados, nos quais uma única configuração é alterada e comparada à configuração de referência, com as demais mantidas fixas, em execuções independentes de um mesmo conjunto de dados. A Tabela 3.5 resume os testes realizados.

O peso da penalidade de capacidade W_{cap}, conforme a Equação (3.12), foi o parâmetro com a calibração mais detalhada, obtido por meio de uma varredura na instância média (conjunto 1). Foram testados dez valores, de 200 a 5000, com três execuções independentes de 60 segundos para cada um e os demais parâmetros mantidos fixos. Para cada execução, registraram-se a função objetivo Z e a violação de capacidade V_{cap} da melhor solução encontrada.

Tabela 3.5: Testes de calibração dos parâmetros do Algoritmo Memético

Teste	**Pergunta**	**Resultado**	**Decisão**
Peso de capacidade	Qual valor de W_{cap} reduz a violação ao longo da busca?	Varredura de 200 a 5000 na instância média, em que 1900 teve o menor Z entre os pesos com violação média abaixo de 1 paciente-dia (Figura 3.1)	$W_{cap} = 1900$
Momento do reparo	Reparar a cada geração ou apenas a população final?	Na instância grande, reparar a cada geração deu Z 9% menor em 30 de 30 pares de execuções. Reparar só após o VND deu Z equivalente, com menos gerações	Reparo antes e depois do VND
Busca local na elite	Refinar também os indivíduos de elite?	Z melhorou 2%, 23% e 45% nas instâncias pequena, média e grande (teste anterior à etapa de reparo)	Adotado
População inicial gulosa	Construir toda a população de forma gulosa?	Com o reparo, Z de 1,3% a 7,9% maior nos quatro portes (10 conjuntos por porte)	Mantidos 20% gulosos
População escalada	Aumentar a população com o porte?	Com 480 indivíduos, a instância grande completou 1 geração e Z quase dobrou em relação a 30 indivíduos	População fixa em 30
Perturbação	Reiniciar parte da população na estagnação?	Sem ganho consistente nas instâncias média e grande	Não adotado
Fração refinada	Refinar menos indivíduos para ter mais gerações?	23 gerações contra 14, mas Z 6% pior	Mantidos 30%
Limite inferior	Montar um modelo separado para a relaxação linear?	Não resolvida em 300 s na grande, e montagem de 289 s na muito grande	Limite reportado pelo *solver*

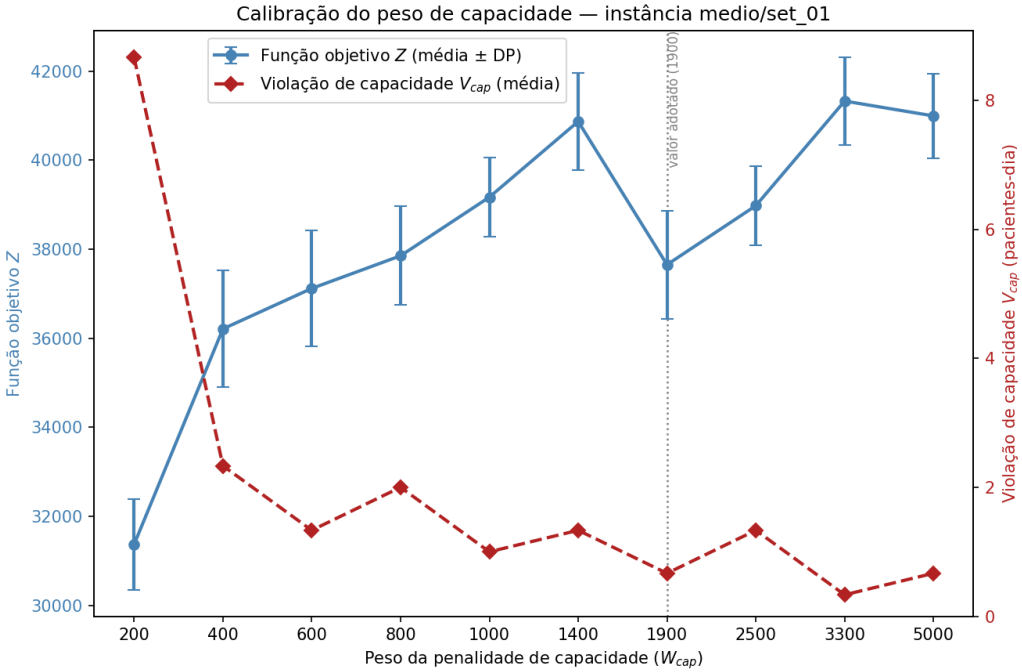


Figura 3.1: Função objetivo Z (média $\pm$ desvio padrão) e violação de capacidade V_{cap} (média) da melhor solução heurística para cada valor de W_{cap} — instância média, conjunto 1, três execuções de 60 s por valor

A Figura 3.1 mostra os dois efeitos opostos do peso. Com $W_{cap} = 200$, a penalidade é fraca o bastante para a busca aceitar exceder a capacidade em troca de um Z menor, e obtém-se o menor Z médio (31 367), mas com 8,7 pacientes-dia sem leito. Ao aumentar o peso, a violação cai rapidamente, enquanto Z tende a crescer.

Esse efeito, porém, deve ser interpretado com cuidado. Realocar um paciente-dia excedente para um quarto com vaga custa, no pior caso, $W_{spec} + W_{gen} + 2W_{transf} = 275$ em Z, conforme a Equação (3.13). Assim, para qualquer W_{cap} acima de 275, toda solução inviável pode ser convertida em uma solução viável de *fitness* menor, e o menor F possível corresponde sempre a uma solução viável. Abaixo desse valor, como em $W_{cap} = 200$, exceder a capacidade pode de fato compensar. Acima dele, o peso não altera o melhor F possível, mas apenas a dinâmica da busca. A varredura confirma essa leitura, pois a violação persiste em todos os pesos testados, inclusive em 3300 e 5000, o que indica que a violação residual se deve à busca não eliminar os últimos excessos, e não a um peso insuficiente. Por esse motivo, em vez de aumentar o peso, adotou-se a etapa de reparo (Subseção 3.3.6).

Como critério de escolha, adotou-se o menor Z entre os pesos que mantêm a violação média abaixo de 1 paciente-dia. Três valores atendem a essa condição, 1900 ($Z = 37\,650$ e $V_{cap} = 0{,}7$), 3300 ($Z = 41\,325$ e $V_{cap} = 0{,}3$) e 5000 ($Z = 40\,992$ e $V_{cap} = 0{,}7$), e, por ter o menor Z entre eles, foi adotado $W_{cap} = 1900$.

Essa escolha tem limitações. Com três execuções por valor, os desvios padrão de Z (cerca de 1 000 a 1 200) se sobrepõem entre os pesos acima de 275, de modo que as diferenças entre 1900, 3300 e 5000 estão dentro do ruído experimental e indicam apenas uma tendência. Além disso, a varredura usou um único conjunto da instância média, com 60 segundos por execução, abaixo do teto de 120 segundos desse porte, e foi realizada antes da adoção da

busca local na elite e da etapa de reparo. Com o reparo, o peso passou a atuar apenas dentro da busca local, pois a população entre gerações é sempre viável. O valor obtido foi, ainda assim, aplicado a todos os portes.

Capítulo 4

Resultados

Este capítulo apresenta a avaliação experimental e comparativa das abordagens exata (Gurobi) e heurística (Algoritmo Memético) aplicadas ao Problema de Alocação de Leitos (PBA), considerando os quatro portes de instâncias definidos no protocolo experimental (Tabela 3.4).

A Seção 4.1 analisa o desempenho relativo entre o *solver* exato e a heurística em cada escala de problema, avaliando o valor da função objetivo Z, conforme a Equação (3.3), a viabilidade das soluções geradas pelo método estocástico e o esforço computacional despendido. A Seção 4.2 examina o impacto operacional das alocações sob a perspectiva da gestão hospitalar, detalhando a adequação de especialidade médica, o volume de transferências internas e a incidência de quartos mistos, além de comparar os portes entre si. Os principais achados são retomados no Capítulo 5.

4.1 Comparação entre o Gurobi e a heurística

Para cada porte de instância, os resultados dos dez conjuntos são apresentados em duas tabelas, uma com os limites do valor ótimo de Z e outra com os resultados da heurística, seguidas de um *boxplot* da dispersão de Z entre as execuções de cada conjunto e das figuras de convergência. As colunas de cada tabela são descritas em sua nota, e, no texto, a execução de menor *fitness* F de cada conjunto é chamada de melhor execução.

No Gurobi, a capacidade dos quartos é uma restrição rígida, enquanto na heurística ela é penalizada no *fitness* durante a busca local, conforme a Equação (3.12), e garantida pela etapa de reparo descrita na Subseção 3.3.6. Por isso, todas as soluções heurísticas reportadas neste capítulo são viáveis para o modelo exato, e a tabela da heurística mantém a coluna V_{cap} apenas para registrar essa verificação.

4.1.1 Instância pequena

Na instância pequena (100 pacientes, 30 segundos por execução), o Gurobi obteve a melhor solução em nove dos dez conjuntos (Tabela 4.1 e Tabela 4.2). A melhor execução da heurística ficou, em média, 1,6% acima da solução incumbente do Gurobi, com variação entre 0,3% abaixo, no conjunto 3, e 4,5% acima, enquanto a média das execuções ficou 4,2% acima. Embora o Gurobi não tenha comprovado a otimalidade, finalizou com *gaps* reduzidos, variando entre 1,0% e 16,0%, com média de 5,9%, o que mostra que o modelo exato é aplicável para esse porte.

Como o Gurobi não comprovou a otimalidade em nenhum conjunto desse porte, a distância das soluções ao ótimo é avaliada por limites inferiores. Além do limite do *solver*, utiliza-se o limite combinatório LB_{comb}, conforme a Equação (3.17), que corresponde ao custo mínimo de especialidade inadequada imposto pelos dados. Com o maior dos dois limites, L, o excesso máximo da heurística sobre o ótimo é $(Z_h - L)/L$, conforme a Equação (3.18), em que Z_h é o valor da melhor execução. Nesse porte, a melhor execução ficou, em média, no máximo 8,2% acima do ótimo, variando de 1,9% a 21,2% conforme o conjunto. Além disso, o limite do Gurobi é sempre maior que LB_{comb}, em média 8,0% acima dele, pois o *solver* avança na árvore de busca e incorpora parte do custo de transferências e de quartos mistos.

A análise da convergência, ilustrada na Figura 4.2 e na Figura 4.3, mostra que cerca de 70% da redução de *fitness* ocorre na primeira geração, quando a busca local (VND) é aplicada pela primeira vez, à elite da população inicial e a parte dos filhos. Como a população inicial já passa pelo reparo, que elimina a violação de capacidade escolhendo as realocações mais baratas, o ponto de partida das curvas já é bem menor que o de uma população sem reparo. A última melhora ocorre, em média, na 12ª geração, de um total de 20,9 gerações. Nas gerações seguintes, o algoritmo estagna em ótimos locais. Na instância pequena, a limitação da heurística é a capacidade de sair de ótimos locais em espaços de busca menores.

Tabela 4.1: Limites do valor ótimo de Z por conjunto — instância pequena

	Gurobi			**Limite combinatório**
Conjunto	**Limite superior**	**Limite inferior**	***Gap***	LB_{comb}
01	11 800,0	11 050,0	6,4%	10 900,0
02	11 975,0	11 850,0	1,0%	11 200,0
03	9 075,0	8 100,0	10,7%	7 400,0
04	5 775,0	5 600,0	3,0%	4 900,0
05	8 525,0	8 025,0	5,9%	7 100,0
06	16 250,0	15 350,0	5,5%	15 200,0
07	5 475,0	4 600,0	16,0%	4 000,0
08	11 125,0	10 575,0	4,9%	10 000,0
09	9 275,0	9 100,0	1,9%	8 200,0
10	14 450,0	13 950,0	3,5%	13 500,0

Nota - *Gap* na convenção do *solver*. LB_{comb}: limite inferior combinatório, conforme a Equação (3.17).

Tabela 4.2: Resultados da heurística por conjunto — instância pequena (10 execuções por conjunto; Z sem a penalidade de capacidade)

Conjunto	Z **da execução de menor** F	V_{cap}	Z **médio**	**CV**	**Excesso máx.**
01	12 225,0	0	12 547,5	1,9%	10,6%
02	12 075,0	0	12 367,5	1,5%	1,9%
03	9 050,0	0	9 302,5	1,8%	11,7%
04	5 825,0	0	5 945,0	2,3%	4,0%
05	8 725,0	0	8 950,0	2,0%	8,7%
06	16 350,0	0	16 450,0	0,5%	6,5%
07	5 575,0	0	5 892,5	3,3%	21,2%
08	11 625,0	0	11 782,5	1,4%	9,9%
09	9 375,0	0	9 675,0	3,2%	3,0%
10	14 600,0	0	14 835,0	1,2%	4,7%

Nota - Z da execução de menor F: valor de Z da execução com menor *fitness*, conforme a Equação (3.12). V_{cap}: violação de capacidade dessa execução, em pacientes-dia, conforme a Equação (3.11), sendo $V_{cap} > 0$ uma solução inviável para o modelo exato. CV: coeficiente de variação de Z entre as execuções (desvio padrão dividido pela média), como medida de dispersão. Excesso máx.: excesso máximo da execução de menor F sobre o ótimo, conforme a Equação (3.18), calculado com o maior entre o limite inferior do Gurobi e LB_{comb}.

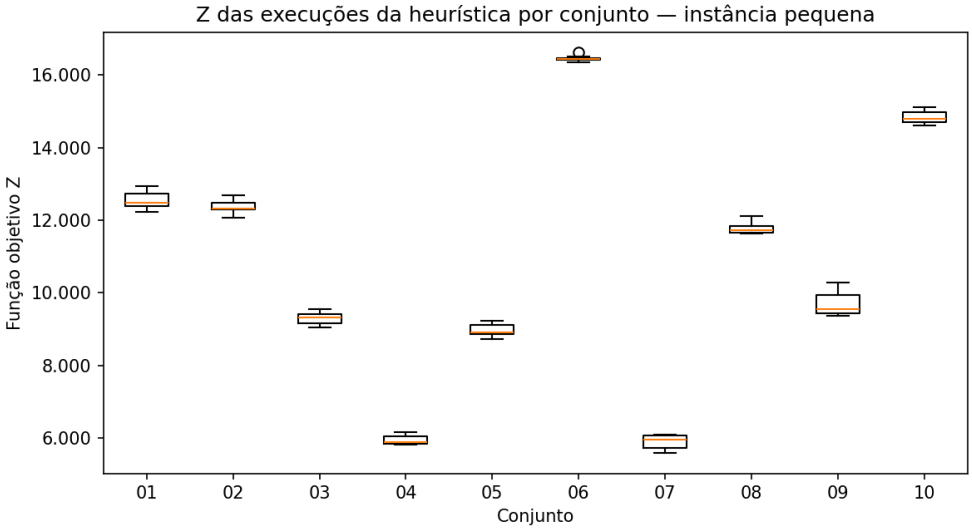


Figura 4.1: Dispersão de Z entre as execuções da heurística em cada conjunto — instância pequena

A Figura 4.1 separa as duas fontes de variação descritas na Seção 3.5. A variação entre as execuções de um mesmo conjunto, devida à aleatoriedade da heurística, é pequena, com coeficiente de variação médio de 1,9%. Já a variação entre os conjuntos, devida ao sorteio das instâncias, é muito maior, com médias de Z entre 5 893 e 16 450 e coeficiente de variação de 32,3% entre elas. A dispersão de Z observada em cada porte se deve, portanto, principalmente às diferenças entre os conjuntos gerados, e não à heurística.

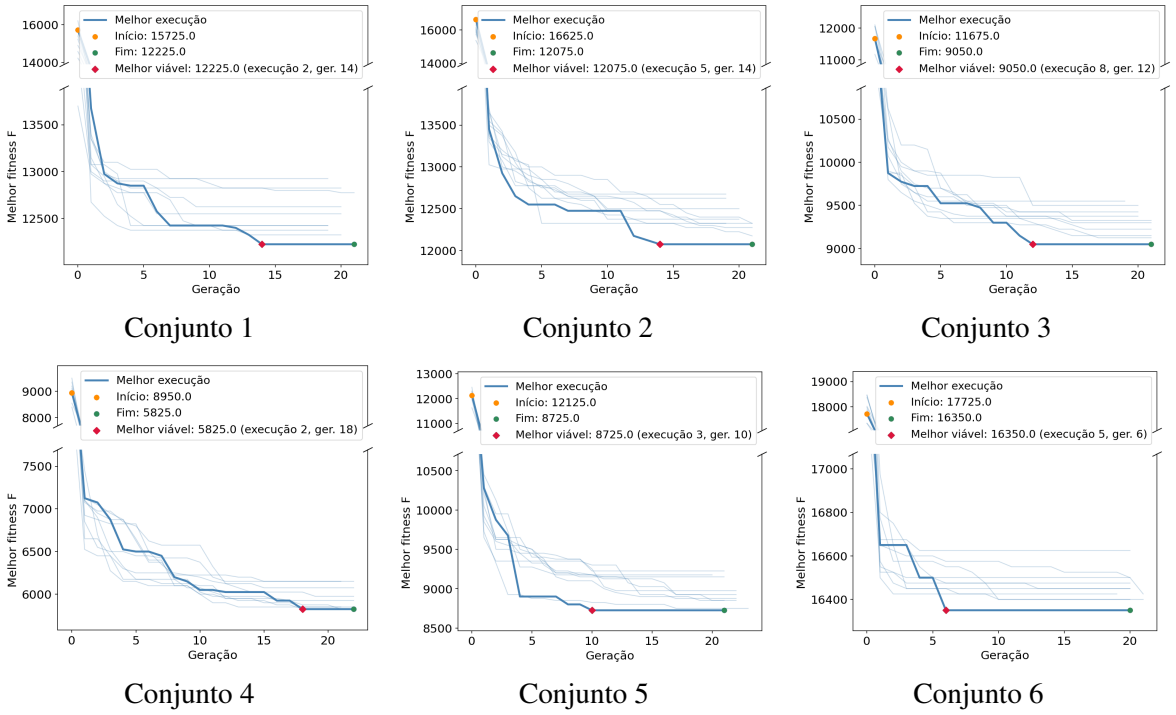


Figura 4.2: Convergência da heurística, com eixo vertical quebrado, nos conjuntos 1 a 6 — instância pequena

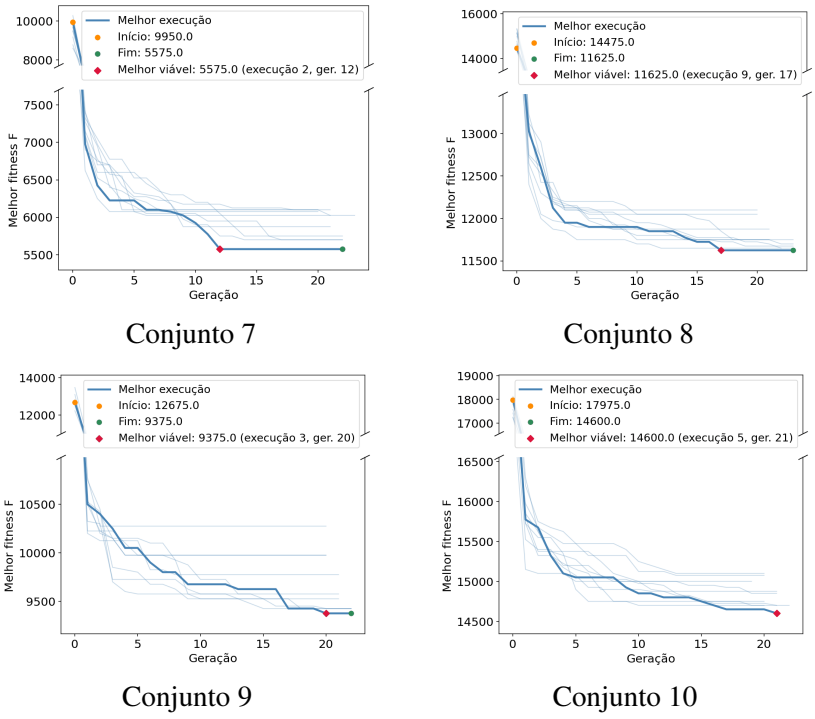


Figura 4.3: Convergência da heurística, com eixo vertical quebrado, nos conjuntos 7 a 10 — instância pequena

4.1.2 Instância média

Na instância média (400 pacientes, 120 segundos por execução), ocorre a inversão entre as abordagens, e a heurística obteve o menor Z em todos os 10 conjuntos (Tabela 4.3 e Tabela 4.4), com redução média de 66,1% em relação à solução incumbente do Gurobi sob o mesmo teto de tempo (de 55,4% a 77,7%). A vantagem não depende de uma execução favorável, pois a média das execuções também apresentou redução de 65,2%. Essa redução compara a heurística com a melhor solução que o Gurobi encontrou no tempo disponível, e não com o ótimo. A partir desse porte, portanto, ela mede sobretudo a dificuldade do método exato, e não a qualidade da heurística, que é avaliada a seguir pelo excesso sobre o limite inferior.

Com quatro vezes mais pacientes que a instância pequena, o Gurobi finalizou com *gaps* de 60,5% a 81,3%. Esses *gaps* elevados decorrem sobretudo da solução incumbente, e não do limite inferior, que continua útil e confirma a qualidade da heurística, já que a melhor execução está, em média, no máximo 19,3% acima do ótimo (de 12,9% a 27,3%). Logo, o próprio limite inferior está a no máximo essa distância do ótimo, e, como ele nunca supera o ótimo, a distância real da heurística é menor ou igual a esses valores. Nesse porte, o limite do Gurobi supera LB_{comb} em apenas 1,9%, em média (de 0,3% a 3,8%), ou seja, o limite obtido pelo *solver* corresponde essencialmente ao mínimo de alocações em especialidade inadequada imposto pelos dados, e pouco acrescenta quanto às transferências e aos quartos mistos.

Ao contrário da instância pequena, as curvas de convergência (Figura 4.5 e Figura 4.6) ainda apresentam potencial de melhoria, sendo que a última melhora ocorre, em média, na geração 13,7 de 14,7, e 28 das 100 execuções ainda melhoraram na última geração. Nesse porte, a heurística é limitada pelo tempo disponível, e não pela estagnação.

Tabela 4.3: Limites do valor ótimo de Z por conjunto — instância média

	Gurobi			**Limite combinatório**
Conjunto	**Limite superior**	**Limite inferior**	***Gap***	LB_{comb}
01	79 300,0	22 350,0	71,8%	21 800,0
02	73 625,0	18 000,0	75,6%	17 500,0
03	91 800,0	27 400,0	70,2%	27 200,0
04	84 525,0	23 800,0	71,8%	23 500,0
05	89 950,0	20 450,0	77,3%	19 900,0
06	70 050,0	24 550,0	65,0%	24 400,0
07	83 950,0	22 750,0	72,9%	22 400,0
08	88 275,0	16 500,0	81,3%	15 900,0
09	92 650,0	29 400,0	68,3%	28 700,0
10	86 775,0	34 300,0	60,5%	34 200,0

Nota - *Gap* na convenção do *solver*. LB_{comb}: limite inferior combinatório, conforme a Equação (3.17).

Tabela 4.4: Resultados da heurística por conjunto — instância média (10 execuções por conjunto; Z sem a penalidade de capacidade)

Conjunto	Z **da execução de menor** F	V_{cap}	Z **médio**	**CV**	**Excesso máx.**
01	28 450,0	0	29 132,5	1,7%	27,3%
02	22 525,0	0	23 240,0	2,2%	25,1%
03	31 925,0	0	32 662,5	1,6%	16,5%
04	28 050,0	0	28 792,5	1,5%	17,9%
05	25 250,0	0	25 742,5	1,3%	23,5%
06	29 050,0	0	29 547,5	1,3%	18,3%
07	26 850,0	0	27 912,5	2,5%	18,0%
08	19 700,0	0	20 660,0	2,3%	19,4%
09	33 650,0	0	34 352,5	1,1%	14,5%
10	38 725,0	0	39 435,0	1,3%	12,9%

Nota - Z da execução de menor F: valor de Z da execução com menor *fitness*, conforme a Equação (3.12). V_{cap}: violação de capacidade dessa execução, em pacientes-dia, conforme a Equação (3.11), sendo $V_{cap} > 0$ uma solução inviável para o modelo exato. CV: coeficiente de variação de Z entre as execuções (desvio padrão dividido pela média), como medida de dispersão. Excesso máx.: excesso máximo da execução de menor F sobre o ótimo, conforme a Equação (3.18), calculado com o maior entre o limite inferior do Gurobi e LB_{comb}.

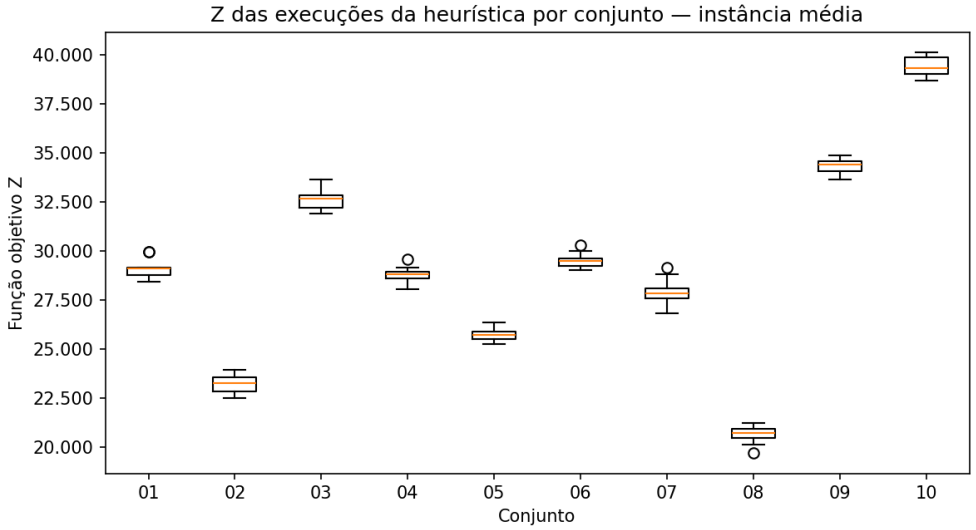


Figura 4.4: Dispersão de Z entre as execuções da heurística em cada conjunto — instância média

Assim como na instância pequena, a Figura 4.4 mostra que a variação entre as execuções de um mesmo conjunto, com coeficiente de variação médio de 1,7%, é bem menor que a variação entre os conjuntos, cujas médias de Z vão de 20 660 a 39 435, com coeficiente de variação de 18,6%.

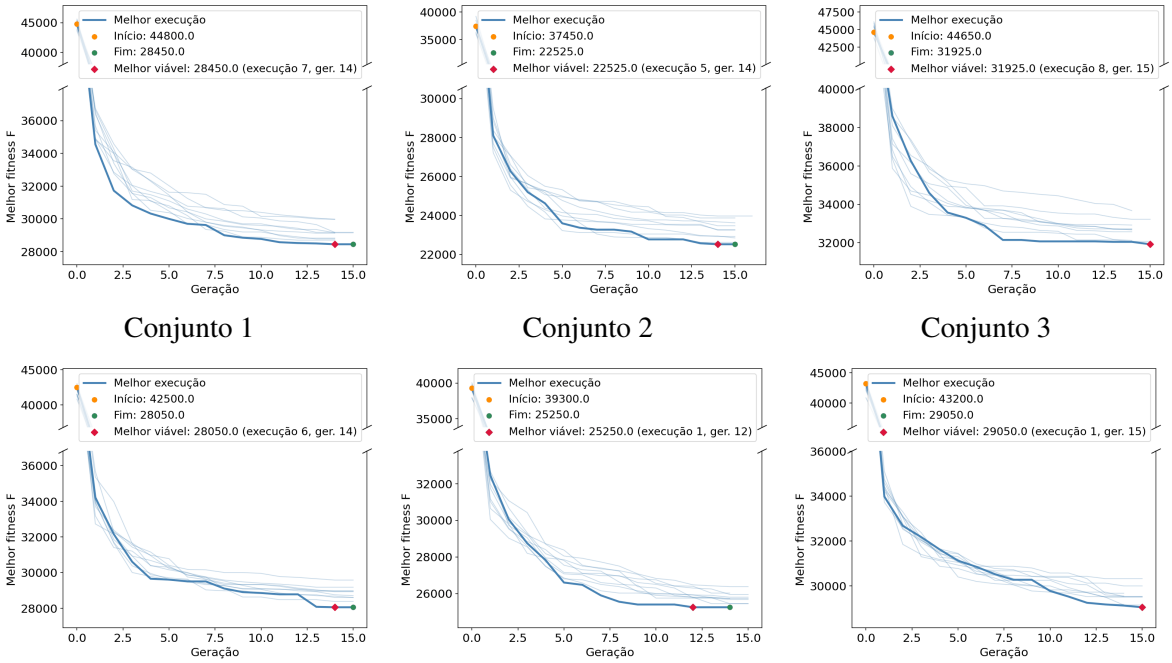


Figura 4.5: Convergência da heurística, com eixo vertical quebrado, nos conjuntos 1 a 6 — instância média

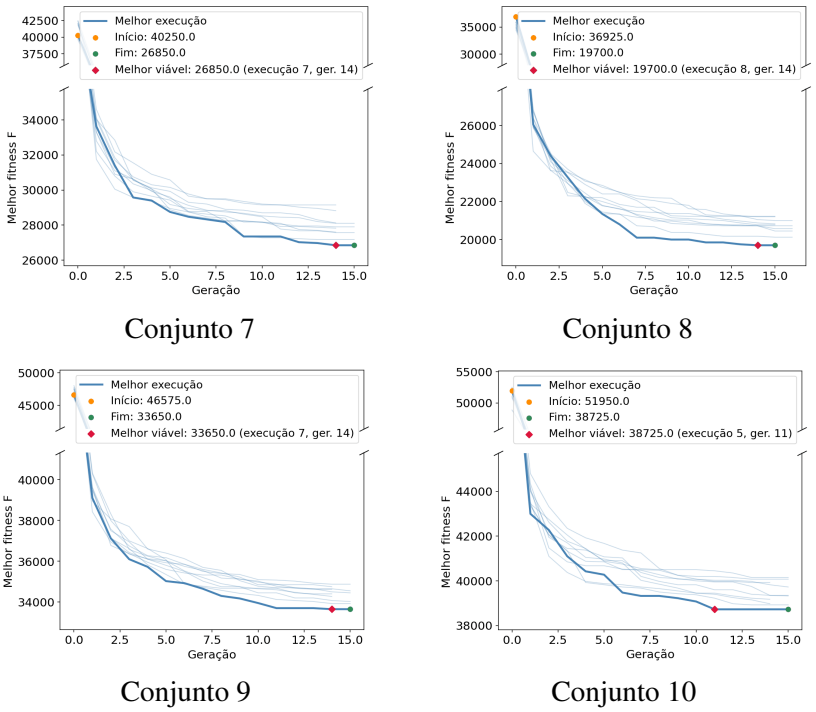


Figura 4.6: Convergência da heurística, com eixo vertical quebrado, nos conjuntos 7 a 10 — instância média

4.1.3 Instância grande

Na instância grande (1600 pacientes, 480 segundos por execução), a diferença entre as abordagens aumenta. A melhor execução da heurística reduziu Z, em média, em 93,9% em relação à solução incumbente do Gurobi (de 91,7% a 95,7%), com vantagem em todos os conjuntos (Tabela 4.5 e Tabela 4.6). Em 8 dos 10 conjuntos, o Gurobi não concluiu a relaxação inicial e terminou com $LB = 0$, o limite trivial. Nesses conjuntos, LB_{comb} é o único limite não trivial disponível, e, nos conjuntos 3 e 6, os dois limites são próximos, com o do Gurobi ligeiramente maior. Com o maior deles, a melhor execução da heurística está, em média, no máximo 76,7% acima do ótimo (de 41,9% a 155,0%). Esses valores são altos porque LB_{comb} considera apenas a parcela de especialidade, enquanto, nesse porte, as transferências e os quartos mistos respondem por cerca de 40% do Z da heurística. O limite, portanto, é fraco demais para atestar a qualidade da solução heurística, e mostra sobretudo que o Gurobi está longe de resolver o problema, já que o seu limite superior é, em média, cerca de 32 vezes LB_{comb}. Esse comportamento é coerente com a literatura, pois Demeester et al. [8] não obtiveram sequer uma solução viável em uma hora de execução de um *solver* comercial em instâncias de tamanho real, e o método exato de Ceschia e Schaerf [6] resolveu apenas instâncias de até 250 pacientes.

As curvas de convergência (Figura 4.8 e Figura 4.9) repetem o comportamento da instância média, com melhora até as últimas gerações, e 74 das 100 execuções ainda melhoraram na última delas.

Tabela 4.5: Limites do valor ótimo de Z por conjunto — instância grande

	Gurobi			**Limite combinatório**
Conjunto	**Limite superior**	**Limite inferior**	***Gap***	LB_{comb}
01	811 250,0	0,0	—	30 200,0
02	819 575,0	0,0	—	40 800,0
03	820 500,0	47 700,0	94,2%	47 300,0
04	830 875,0	0,0	—	15 500,0
05	816 925,0	0,0	—	32 700,0
06	815 200,0	29 650,0	96,4%	28 600,0
07	821 300,0	0,0	—	13 900,0
08	835 450,0	0,0	—	42 100,0
09	809 600,0	0,0	—	20 000,0
10	822 025,0	0,0	—	30 300,0

Nota - — indica que o *solver* não obteve limite inferior não trivial dentro do teto de tempo. *Gap* na convenção do *solver*. LB_{comb}: limite inferior combinatório, conforme a Equação (3.17).

Tabela 4.6: Resultados da heurística por conjunto — instância grande (10 execuções por conjunto; Z sem a penalidade de capacidade)

Conjunto	Z **da execução de menor** F	V_{cap}	Z **médio**	**CV**	**Excesso máx.**
01	50 050,0	0	51 685,0	1,5%	65,7%
02	62 500,0	0	63 560,0	1,5%	53,2%
03	67 700,0	0	68 612,5	0,9%	41,9%
04	37 825,0	0	39 185,0	1,7%	144,0%
05	52 550,0	0	53 830,0	1,3%	60,7%
06	46 700,0	0	48 522,5	1,7%	57,5%
07	35 450,0	0	37 377,5	2,5%	155,0%
08	60 975,0	0	62 785,0	1,5%	44,8%
09	35 975,0	0	37 415,0	2,0%	79,9%
10	49 650,0	0	51 357,5	1,9%	63,9%

Nota - Z da execução de menor F: valor de Z da execução com menor *fitness*, conforme a Equação (3.12). V_{cap}: violação de capacidade dessa execução, em pacientes-dia, conforme a Equação (3.11), sendo $V_{cap} > 0$ uma solução inviável para o modelo exato. CV: coeficiente de variação de Z entre as execuções (desvio padrão dividido pela média), como medida de dispersão. Excesso máx.: excesso máximo da execução de menor F sobre o ótimo, conforme a Equação (3.18), calculado com o maior entre o limite inferior do Gurobi e LB_{comb}.

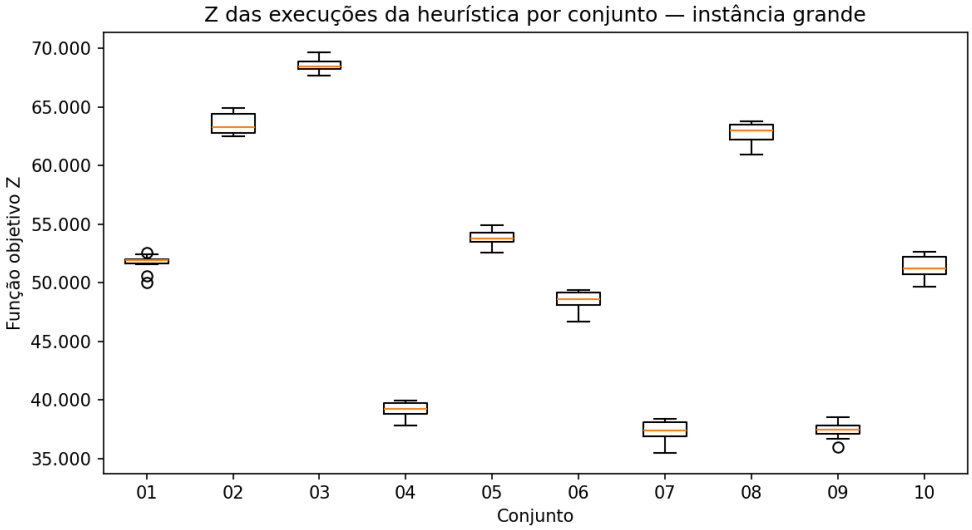


Figura 4.7: Dispersão de Z entre as execuções da heurística em cada conjunto — instância grande

Assim como nas instâncias anteriores, a Figura 4.7 mostra que a variação entre as execuções de um mesmo conjunto (coeficiente de variação médio de 1,6%) é bem menor que a variação entre os conjuntos (médias de Z de 37 378 a 68 612, coeficiente de variação de 21,8%).

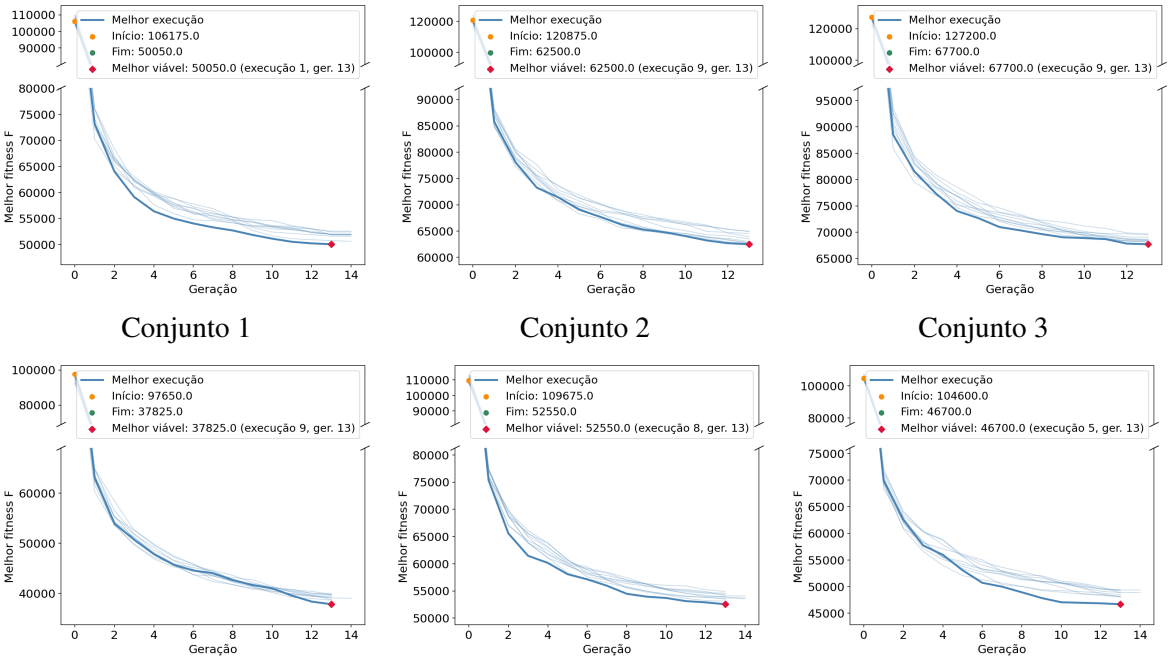


Figura 4.8: Convergência da heurística, com eixo vertical quebrado, nos conjuntos 1 a 6 — instância grande

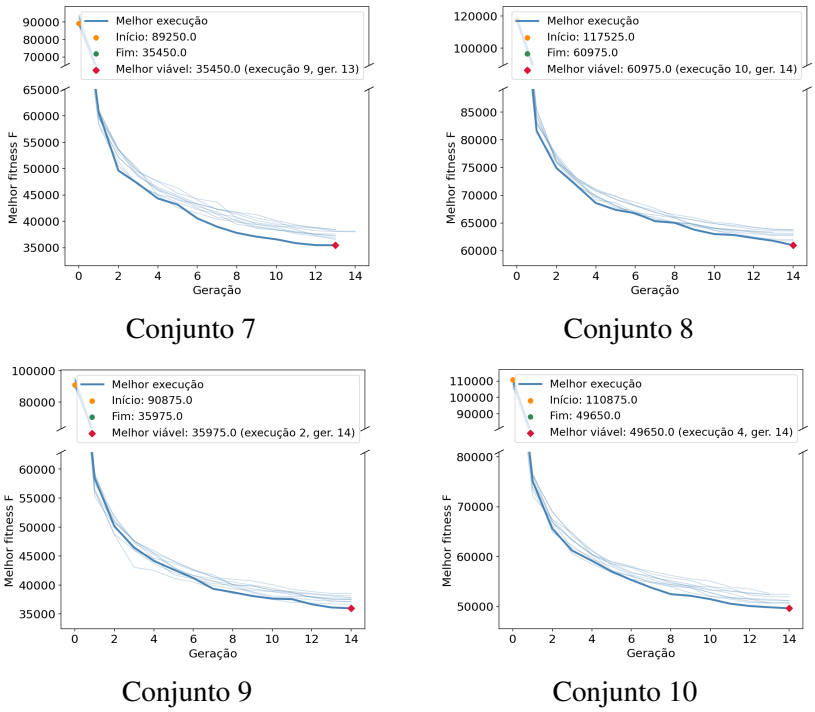


Figura 4.9: Convergência da heurística, com eixo vertical quebrado, nos conjuntos 7 a 10 — instância grande

4.1.4 Instância muito grande

Na instância muito grande (6400 pacientes, 1920 segundos por execução), a heurística novamente obteve o menor Z em todos os conjuntos, com redução média de 95,2% em relação à solução incumbente do Gurobi (de 93,5% a 96,2%), como mostram a Tabela 4.7 e a Tabela 4.8. O Gurobi terminou todos os conjuntos com LB nulo, sem concluir a relaxação inicial, como antecipado na Subseção 2.2.1, de modo que LB_{comb} é o único limite inferior disponível. Com ele, a melhor execução da heurística está, em média, no máximo 120,0% acima do ótimo (de 64,2% a 181,2%). Como na instância grande, o limite é fraco, pois ignora as transferências e os quartos mistos, que nesse porte somam cerca de metade do Z da heurística.

As curvas de convergência (Figura 4.11 e Figura 4.12) ainda melhoram na última geração em 93 das 100 execuções. Com 10,5 gerações por execução, contra 13 a 15 nas instâncias média e grande, este é o porte em que a heurística dispõe de menos gerações, embora o teto de tempo acompanhe o número de pacientes.

Tabela 4.7: Limites do valor ótimo de Z por conjunto — instância muito grande

	Gurobi			**Limite combinatório**
Conjunto	**Limite superior**	**Limite inferior**	***Gap***	LB_{comb}
01	3 273 725,0	0,0	—	65 900,0
02	3 276 950,0	0,0	—	67 900,0
03	3 295 100,0	0,0	—	74 300,0
04	3 289 725,0	0,0	—	100 400,0
05	3 272 475,0	0,0	—	76 900,0
06	3 257 200,0	0,0	—	43 900,0
07	3 284 350,0	0,0	—	65 000,0
08	3 261 000,0	0,0	—	45 400,0
09	3 297 800,0	0,0	—	131 000,0
10	3 263 375,0	0,0	—	85 300,0

Nota - — indica que o *solver* não obteve limite inferior não trivial dentro do teto de tempo. *Gap* na convenção do *solver*. LB_{comb}: limite inferior combinatório, conforme a Equação (3.17).

Tabela 4.8: Resultados da heurística por conjunto — instância muito grande (10 execuções por conjunto; Z sem a penalidade de capacidade)

Conjunto	Z **da execução de menor** F	V_{cap}	Z **médio**	**CV**	**Excesso máx.**
01	140 350,0	0	142 217,5	1,0%	113,0%
02	156 950,0	0	159 045,0	0,9%	131,1%
03	161 300,0	0	163 847,5	1,0%	117,1%
04	184 550,0	0	187 877,5	1,0%	83,8%
05	163 575,0	0	165 377,5	1,1%	112,7%
06	123 425,0	0	125 275,0	1,0%	181,2%
07	147 325,0	0	151 670,0	1,9%	126,7%
08	126 525,0	0	128 665,0	1,0%	178,7%
09	215 125,0	0	218 490,0	0,8%	64,2%
10	163 125,0	0	167 347,5	1,2%	91,2%

Nota - Z da execução de menor F: valor de Z da execução com menor *fitness*, conforme a Equação (3.12). V_{cap}: violação de capacidade dessa execução, em pacientes-dia, conforme a Equação (3.11), sendo $V_{cap} > 0$ uma solução inviável para o modelo exato. CV: coeficiente de variação de Z entre as execuções (desvio padrão dividido pela média), como medida de dispersão. Excesso máx.: excesso máximo da execução de menor F sobre o ótimo, conforme a Equação (3.18), calculado com o maior entre o limite inferior do Gurobi e LB_{comb}.

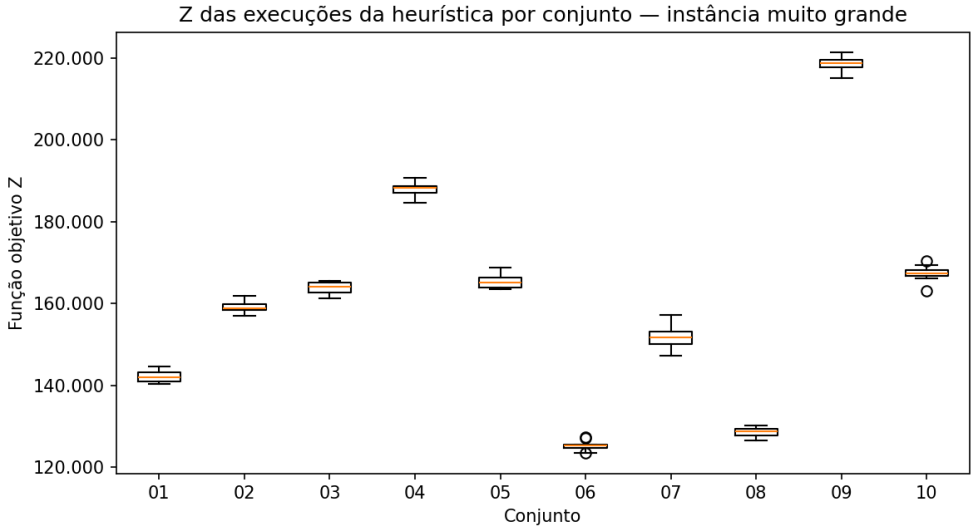


Figura 4.10: Dispersão de Z entre as execuções da heurística em cada conjunto — instância muito grande

Também nesse porte, a Figura 4.10 mostra que a variação entre as execuções (coeficiente de variação médio de 1,1%) é bem menor que a variação entre os conjuntos (médias de Z de 125 275 a 218 490, coeficiente de variação de 17,2%).

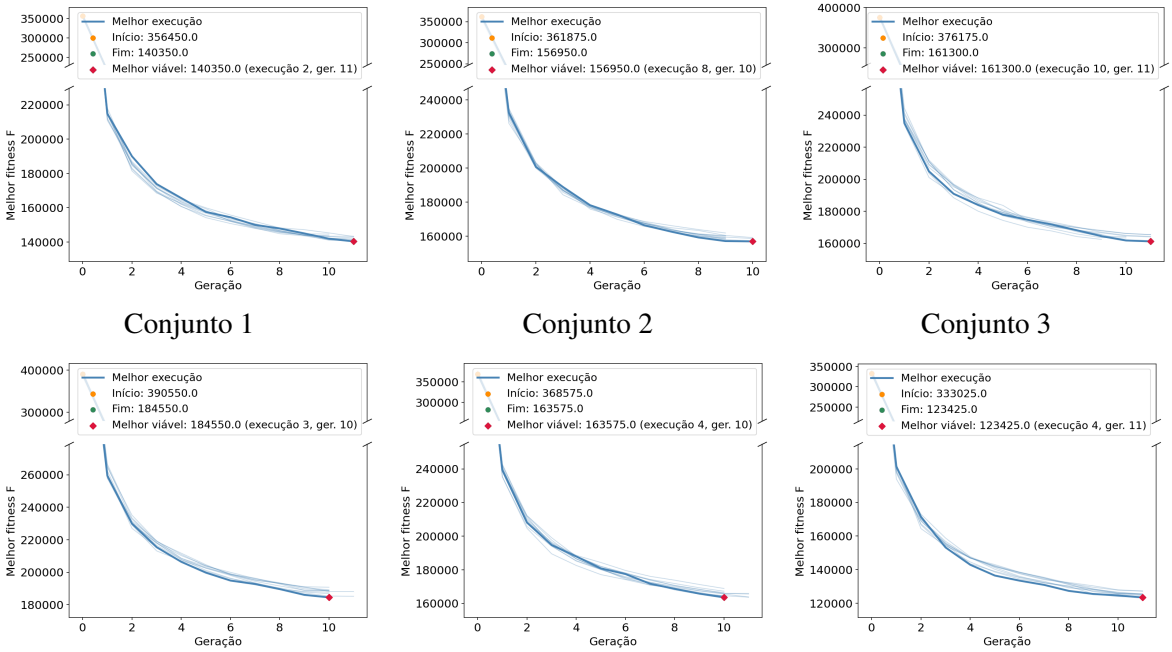


Figura 4.11: Convergência da heurística, com eixo vertical quebrado, nos conjuntos 1 a 6 — instância muito grande

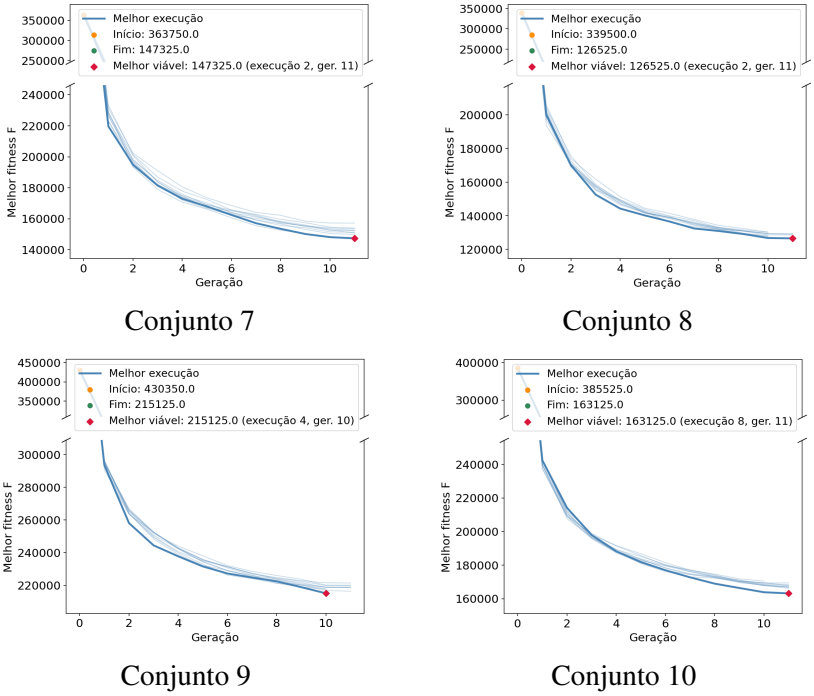


Figura 4.12: Convergência da heurística, com eixo vertical quebrado, nos conjuntos 7 a 10 — instância muito grande

4.1.5 Viabilidade e custo do reparo

Como a heurística trata a capacidade como restrição flexível durante a busca local, a viabilidade de suas soluções é verificada à parte. A Tabela 4.9 resume, por porte, quantas execuções terminaram com solução viável e a violação de capacidade da melhor solução de cada conjunto, medida em pacientes-dia pelo V_{cap}, conforme a Equação (3.11), e em relação à carga de internação pela taxa ρ_{cap}, conforme a Equação (3.15). Com a etapa de reparo (Subseção 3.3.6), as 400 execuções terminaram com soluções viáveis, sem excesso de ocupação em nenhum quarto e em nenhum dia.

Tabela 4.9: Viabilidade das soluções heurísticas por porte (melhor solução de cada conjunto, média $\pm$ desvio padrão entre os 10 conjuntos)

Porte	**Execuções que encontraram solução viável**	**Nº Violações de capacidade (V_{cap})**	**Carga de internação (N_{pd})**	**Taxa de violação de capacidade ($\rho_{cap} = V_{cap}/N_{pd}$)**
Pequena	100/100	0,0 $\pm$ 0,0	397,8 $\pm$ 11,0	0,000% $\pm$ 0,000%
Média	100/100	0,0 $\pm$ 0,0	1 611,6 $\pm$ 21,1	0,000% $\pm$ 0,000%
Grande	100/100	0,0 $\pm$ 0,0	6 420,6 $\pm$ 41,7	0,000% $\pm$ 0,000%
Muito grande	100/100	0,0 $\pm$ 0,0	25 601,0 $\pm$ 107,7	0,000% $\pm$ 0,000%

As soluções do Gurobi também são sempre viáveis, mas a sua qualidade se deteriora rapidamente com o porte. Calculada em cada conjunto e depois resumida pela média, a razão entre o Z do Gurobi e o da melhor execução da heurística já é de 3,1 na instância média (de 2,2 a 4,5). Essa razão cresce para 17,2 na instância grande (de 12,1 a 23,2) e para 21,2 na muito grande (de 15,3 a 26,4). A diferença cresce pouco da instância grande para a muito grande porque, já na grande, a solução do Gurobi equivale a uma alocação feita ao acaso (Seção 4.2), cujo custo por paciente-dia não aumenta com o porte.

O reparo tem um custo em tempo, descontado do mesmo teto concedido ao Gurobi. A fração do teto gasta reparando indivíduos cresce com o porte, de 7,6% na instância pequena para 20,3%, 24,8% e 26,1% nas instâncias média, grande e muito grande, e se concentra nos indivíduos gerados pelo construtor aleatório, na população inicial e nos imigrantes, que partem de violações de centenas a milhares de pacientes-dia.

Esse custo, porém, é compensado pela melhora da busca. Em uma execução anterior do mesmo protocolo, com a mesma heurística sem a etapa de reparo, apenas 5 das 100 execuções da instância grande e nenhuma da muito grande encontraram uma solução viável, e a melhor solução violou a capacidade em 7 e em 10 dos 10 conjuntos, respectivamente, ainda que em menos de 0,1% da carga de internação. Em comparação com essa versão, o Z médio das execuções com reparo foi 1,3%, 1,2%, 6,1% e 13,7% menor nas instâncias pequena, média, grande e muito grande, com redução em 8, 8, 10 e 10 dos 10 conjuntos, respectivamente. A comparação é conservadora, pois a versão com reparo foi executada com mais processos simultâneos na mesma máquina (Subseção 3.4.2) e realizou, por isso, de 3% a 14% menos avaliações por execução. O ganho vem principalmente das transferências, que caíram de 10% a 24% conforme o porte, e, na instância muito grande, também da especialidade inadequada, que caiu 15%. Uma explicação é que, sem o reparo, os filhos chegam ao VND com excessos de capacidade que a busca local desfaz movimento a movimento, muitas vezes com mudanças parciais de quarto (N_3) que criam transferências, enquanto o reparo elimina esses excessos pelas realocações mais baratas e deixa o esforço do VND para a redução de

Z. Em contrapartida, os quartos-dia mistos aumentaram de 9% a 15% nas instâncias média, grande e muito grande, já que a realocação mais barata disponível às vezes coloca pacientes de gêneros distintos no mesmo quarto.

4.1.6 Esforço computacional

Conforme descrito na Subseção 3.4.2, a heurística é executada em um único núcleo, enquanto o Gurobi utiliza, por padrão, todos os núcleos disponíveis.

A Tabela 4.10 compara o esforço de busca das duas abordagens dentro do mesmo teto de tempo, medido pelo número de iterações do simplex no Gurobi e pelo número de avaliações da função objetivo na heurística. As duas medidas não têm a mesma unidade, mas a forma como variam com o porte é comparável.

O esforço do Gurobi não acompanha o crescimento das instâncias. Com o modelo cada vez maior, cada iteração se torna mais custosa e, na instância muito grande, o *solver* realizou apenas 1 334 iterações em 1920 segundos, sem concluir a relaxação inicial. A heurística, por sua vez, manteve uma taxa de 43 a 55 mil avaliações por segundo, incluindo as do reparo, chegando a 82,1 milhões de avaliações por execução na instância muito grande. Essa diferença decorre da avaliação procedural e incremental descrita na Seção 3.4, em que o custo de cada movimento depende apenas da internação do paciente movido, e não do tamanho do modelo.

O número de gerações, por outro lado, cai com o porte, de 20,9 na instância pequena para 10,5 na muito grande, pois, embora o teto de tempo cresça na mesma proporção que o número de pacientes, cada geração exige refinar e reparar indivíduos com cada vez mais pacientes e se torna proporcionalmente mais custosa. Combinado com a convergência discutida nas subseções anteriores, isso explica os dois regimes observados. Na instância pequena, sobram gerações e a heurística estagna, enquanto nas demais as gerações se esgotam enquanto a busca ainda melhora.

Tabela 4.10: Esforço computacional por porte (média por execução)

Porte	Teto	Nº Iterações do Gurobi	Nº Avaliações da heurística	Nº Gerações
Pequena	30 s	771 234	1 643 274	20,9
Média	120 s	51 233	6 493 237	14,7
Grande	480 s	258 168	25 852 469	13,4
Muito grande	1920 s	1 334	82 093 944	10,5

4.2 Discussão das métricas operacionais

Enquanto a Seção 4.1 compara as abordagens pelo valor agregado de Z, esta seção analisa, critério a critério, o que as soluções representam para o hospital. Para cada critério, apresentam-se o melhor resultado possível dado o cenário, quando há um limite conhecido, o que a heurística entregou e o que isso significa para a gestão dos leitos. Todas as métricas são normalizadas dentro de cada conjunto, como taxas, e só então resumidas por porte, por média e desvio padrão entre os 10 conjuntos. Dessa forma, o desvio padrão indica se a heurística se comporta de forma consistente entre os conjuntos ou se acerta em alguns e erra em outros. As características das instâncias, como o horizonte e a ocupação, foram descritas na Subseção 3.1.5.

As tabelas desta seção reportam as métricas do Gurobi apenas na instância pequena, em que o *solver* termina com *gap* de 1% a 16% e sua solução está próxima do ótimo, sendo, portanto, um termo de comparação legítimo. A partir da instância média, a solução do Gurobi é uma incumbente ainda distante do ótimo, com *gap* de 60,5% a 81,3% na instância média e sem limite inferior não trivial do *solver* na maioria dos conjuntos das instâncias grande e muito grande, e suas métricas são muito piores que as da heurística. Nas instâncias grande e muito grande, por exemplo, cerca de 72% dos pacientes-dia estão em especialidade inadequada, há quase três transferências por paciente e cerca de 66% dos quartos-dia são mistos. Esses valores coincidem com os de uma alocação feita ao acaso, independentemente a cada dia. Com a mesma distribuição de especialidades para pacientes e quartos, um quarto sorteado tem especialidade inadequada com probabilidade de $1 - (0{,}4^2 + 3 \cdot 0{,}2^2) = 72\%$, exatamente a taxa observada. Da mesma forma, quase três transferências por paciente equivalem a uma mudança de quarto a quase todo dia, já que o tempo médio de permanência é de quatro dias, e a proporção de quartos mistos é a esperada quando os gêneros são distribuídos ao acaso em quartos de dois ou três pacientes. Isso indica que, nesses portes, o Gurobi encontrou apenas uma solução que respeita a atribuição única e a capacidade, mas praticamente não chegou a otimizar a função objetivo dentro do teto de tempo. Na instância média, os valores são intermediários, pois o *solver* chegou a melhorar parcialmente essa solução. Como essas métricas refletem o fato de o *solver* não ter convergido, e não a qualidade do modelo exato, elas são omitidas das tabelas a partir da instância média, e a heurística é comparada, nesses portes, com os limites mínimos de cada critério.

4.2.1 Especialidade

No critério de especialidade, o melhor resultado possível é conhecido. Denomina-se mínimo estrutural o número mínimo de pacientes-dia em especialidade inadequada, M, conforme a Equação (3.16), que qualquer solução viável necessariamente apresenta, por depender apenas da distribuição dos quartos e da demanda de cada especialidade, e não da alocação. A Tabela 4.11 compara esse mínimo, como porcentagem da carga de internação, com o resultado da heurística.

Tabela 4.11: Especialidade inadequada por porte (% dos pacientes-dia, média ± desvio padrão entre os 10 conjuntos)

Porte	**Mínimo estrutural (M/N_{pd})**	**Heurística**	**Folga da heurística (p.p.)**	**Gurobi**
Pequena	23,2 ± 9,1	23,5 ± 8,9	0,3 ± 0,5	23,4 ± 9,0
Média	14,6 ± 3,5	14,8 ± 3,4	0,2 ± 0,2	—
Grande	4,7 ± 1,7	5,0 ± 1,7	0,3 ± 0,1	—
Muito grande	3,0 ± 1,0	3,1 ± 1,0	0,2 ± 0,1	—

Nota - Folga calculada em cada conjunto, como a taxa da heurística menos o mínimo estrutural do próprio conjunto, e só então agregada. Gurobi reportado apenas na instância pequena. Nas demais, sua solução incumbente está distante do ótimo e não é um termo de comparação legítimo (Seção 4.2).

A heurística deixou 23,5%, 14,8%, 5,0% e 3,1% dos pacientes-dia em especialidade inadequada nas instâncias pequena, média, grande e muito grande, valores que superam o

mínimo estrutural em apenas 0,3, 0,2, 0,3 e 0,2 ponto percentual, respectivamente. O desvio padrão dessa folga, entre 0,1 e 0,5 ponto percentual, mostra que a heurística fica próxima do mínimo em todos os conjuntos, e não apenas em média. Na instância pequena, o Gurobi obteve 23,4%, praticamente o mesmo valor. A folga não cresce com o porte, apesar da redução do número de gerações disponíveis nas instâncias maiores (Subseção 4.1.6).

Para o hospital, isso significa que quase toda a alocação em especialidade inadequada é imposta pela distribuição dos quartos, e não pela alocação. Na instância pequena, metade dos conjuntos não tem nenhum quarto de alguma especialidade, e o mínimo estrutural varia de 10% a 39% dos pacientes-dia conforme o conjunto. Reduzir esse valor exigiria alterar a distribuição de especialidades dos quartos, uma decisão de nível estratégico, e não uma alocação melhor.

4.2.2 Transferências

Para as transferências, não há limite inferior não trivial, já que o valor zero é o único mínimo garantido. A Tabela 4.12 apresenta o número de transferências por 100 pacientes.

Tabela 4.12: Transferências por porte (por 100 pacientes, média ± desvio padrão entre os 10 conjuntos)

Porte	Heurística	Gurobi
Pequena	6,4 ± 3,3	3,6 ± 1,7
Média	12,7 ± 2,7	—
Grande	13,8 ± 1,7	—
Muito grande	14,8 ± 1,0	—

Nota - Não há limite inferior não trivial para o número de transferências. Gurobi reportado apenas na instância pequena. Nas demais, sua solução incumbente está distante do ótimo e não é um termo de comparação legítimo (Seção 4.2).

A heurística realizou, em média, 6,4 transferências por 100 pacientes na instância pequena, taxa que sobe para 12,7 na média, 13,8 na grande e 14,8 na muito grande, com desvio padrão entre 1,0 e 3,3. Na instância pequena, o Gurobi realizou 3,6 transferências por 100 pacientes. A diferença, de 2,8 transferências por 100 pacientes, equivale a 140 unidades de Z e corresponde à maior parte da diferença residual de qualidade entre as soluções.

O crescimento com o porte é consequência direta dos pesos da função objetivo. A penalidade de especialidade é cobrada a cada dia de internação ($W_{spec} = 100$ por paciente-dia), enquanto a de transferência é cobrada uma única vez ($W_{transf} = 50$). Com a razão $W_{transf}/W_{spec} = 0{,}5$, transferir um paciente de um quarto de especialidade inadequada para um quarto adequado, quando lhe restam k dias de internação no novo quarto, reduz Z em $k \cdot W_{spec} - W_{transf} = 100k - 50$, valor positivo para qualquer $k \geq 1$. A função objetivo, portanto, pede explicitamente essas transferências, pois, sempre que um leito da especialidade adequada é liberado durante a internação de um paciente mal alocado, transferi-lo melhora a solução.

Para o hospital, isso corresponde a cerca de 14 a 15 transferências a cada 100 pacientes nas instâncias maiores. Esse número não indica uma falha do algoritmo, mas o equilíbrio definido pela calibração dos pesos. Se as transferências forem mais indesejáveis do que esse equilíbrio sugere, por causarem desconforto ao paciente e trabalho adicional à equipe

[8, 2], o ajuste deve ser feito na relação entre W_{transf} e W_{spec}. Para que transferir deixe de compensar quando restam k dias de internação, seria necessário $W_{transf} \geq k \cdot W_{spec}$.

4.2.3 Quartos mistos

Para os quartos mistos, o melhor resultado possível é limitado pelo mínimo por paridade G, conforme a Equação (3.19), que conta os dias de lotação total em que o número de pacientes do gênero feminino não é múltiplo de três. A Tabela 4.13 compara esse mínimo, como porcentagem dos quartos-dia, com o resultado da heurística.

Tabela 4.13: Quartos-dia mistos por porte (% dos quartos-dia, média $\pm$ desvio padrão entre os 10 conjuntos)

Porte	**Mínimo por paridade (G)**	**Heurística**	**Folga da heurística (p.p.)**	**Gurobi**
Pequena	5,2 $\pm$ 1,7	7,6 $\pm$ 2,7	2,5 $\pm$ 2,2	7,9 $\pm$ 2,8
Média	1,4 $\pm$ 0,4	4,4 $\pm$ 1,0	3,0 $\pm$ 1,2	—
Grande	0,3 $\pm$ 0,1	3,9 $\pm$ 0,5	3,6 $\pm$ 0,5	—
Muito grande	0,1 $\pm$ 0,0	4,3 $\pm$ 0,3	4,2 $\pm$ 0,3	—

Nota - Mínimo por paridade conforme a Equação (3.19). Folga calculada em cada conjunto e só então agregada. Gurobi reportado apenas na instância pequena. Nas demais, sua solução incumbente está distante do ótimo e não é um termo de comparação legítimo (Seção 4.2).

A heurística manteve 7,6%, 4,4%, 3,9% e 4,3% dos quartos-dia com pacientes de gêneros distintos nas instâncias pequena, média, grande e muito grande, enquanto o mínimo por paridade é de 5,2%, 1,4%, 0,3% e 0,1%. Na instância pequena, o Gurobi obteve 7,9%, valor próximo ao da heurística. A folga, de 2,5 a 4,2 pontos percentuais, é maior que na especialidade e cresce nas instâncias grande e muito grande, em parte pelo reparo, que às vezes realoca pacientes para quartos com pacientes de outro gênero (Subseção 4.1.5). Além disso, o mínimo por paridade é fraco, pois ignora a interação com a especialidade dos quartos. Separar os gêneros pode exigir alocar pacientes fora de sua especialidade e, como um paciente-dia em especialidade inadequada custa mais que um quarto-dia misto ($W_{spec} = 100$ contra $W_{gen} = 75$), a função objetivo prefere, nesses casos, o quarto misto. Parte da folga corresponde, portanto, a essa troca entre critérios, e não a uma deficiência da busca.

Para o hospital, entre 4% e 8% dos quartos-dia teriam pacientes de gêneros distintos, com os menores valores nas instâncias maiores. Se a separação de gêneros for uma exigência mais rígida da instituição, ela pode ser tratada como restrição ou com um peso W_{gen} maior que W_{spec}, ao custo de mais alocações em especialidade inadequada.

4.2.4 Comparação entre portes

Como Z é uma soma sobre pacientes-dia, quartos-dia e transferências, o seu valor cresce com o porte da instância e não é comparável entre portes (Subseção 3.1.3). Para comparar os portes, a Tabela 4.14 divide Z pela carga de internação N_{pd}, conforme a Equação (3.14). O mínimo estrutural, definido na Subseção 4.2.1, é expresso aqui em unidades de Z por paciente-dia ($W_{spec} \cdot M/N_{pd}$), o que permite separar, no valor de Z, a parcela imposta pelos dados da parcela que depende do método de solução.

Tabela 4.14: Função objetivo por paciente-dia e mínimo estrutural por porte (média ± desvio padrão entre os 10 conjuntos)

Porte	Z/N_{pd} **do Gurobi**	Z/N_{pd} **da heurística**	**Mínimo estrutural** ($W_{spec} \cdot M/N_{pd}$)	**Excedente da heurística sobre o mínimo estrutural**
Pequena	26,1 ± 8,8	26,5 ± 8,9	23,2 ± 9,1	3,3 ± 0,8
Média	52,2 ± 4,9	17,6 ± 3,5	14,6 ± 3,5	3,0 ± 0,4
Grande	127,8 ± 0,6	7,8 ± 1,8	4,7 ± 1,7	3,1 ± 0,3
Muito grande	128,0 ± 0,4	6,2 ± 1,0	3,0 ± 1,0	3,2 ± 0,2

Nota - valores por paciente-dia. O excedente é calculado em cada conjunto, como Z/N_{pd} da heurística menos o mínimo estrutural do próprio conjunto, e só então resumido por porte.

Por paciente-dia, o Z da heurística cai de 26,5 na instância pequena para 6,2 na muito grande. Essa queda, porém, acompanha quase integralmente a do mínimo estrutural, que passa de 23,2 para 3,0, pois, com mais quartos, a distribuição sorteada das especialidades se aproxima da distribuição da demanda. A grande variação desses dois valores entre os conjuntos de um mesmo porte, com desvio padrão de até 9,1 na instância pequena, tem a mesma origem, já que o mínimo estrutural depende do sorteio das especialidades dos quartos de cada conjunto.

O excedente da heurística sobre o mínimo estrutural, calculado em cada conjunto e só depois resumido por porte, é estável entre os conjuntos, com desvio padrão entre 0,2 e 0,8, e praticamente não varia com o porte, de 3,0 a 3,3 por paciente-dia, mesmo na instância muito grande, em que a heurística dispõe de menos gerações (Subseção 4.1.6). No Gurobi, ao contrário, o Z por paciente-dia cresce de 26,1 para 128,0, refletindo a dificuldade do *solver* nessas instâncias. Dessa forma, a melhora aparente da heurística com o porte se deve principalmente aos dados, e não ao método.

Capítulo 5

Conclusão

Este trabalho tratou o Problema de Alocação de Leitos (PBA) em sua versão voltada ao planejamento tático do censo hospitalar, na qual um conjunto conhecido de pacientes, com datas de admissão e de alta definidas, deve ser alocado aos quartos do hospital ao longo de um horizonte. O problema foi formulado como um modelo de Programação Linear Inteira com função objetivo dada pela soma ponderada de penalidades por especialidade inadequada, transferências e quartos mistos, e resolvido por duas abordagens. A abordagem exata aplicou o *solver* Gurobi ao modelo completo, e a abordagem heurística utilizou um Algoritmo Memético, que combina um Algoritmo Genético com busca local VND e trata a capacidade dos quartos como restrição flexível na busca local, com uma etapa de reparo que mantém a população viável a cada geração. As duas abordagens foram comparadas em instâncias sintéticas de quatro portes, com dez conjuntos independentes por porte e o mesmo teto de tempo para ambas.

Os resultados mostraram dois comportamentos distintos. Na instância pequena, com 100 pacientes, o modelo exato é tratável. O Gurobi obteve a melhor solução em nove dos dez conjuntos, com *gap* de 1% a 16%, e a heurística ficou, em média, 1,6% acima dele e no máximo 8,2% acima do ótimo. A partir da instância média, com 400 pacientes, a situação se inverte. O Gurobi deixa de fechar o *gap* dentro do teto de tempo e, nas instâncias grande e muito grande, não chega a concluir a relaxação inicial, enquanto a heurística obtém o menor Z em todos os 30 conjuntos, com redução de 55% a 96% em relação à solução incumbente do *solver*, sempre com soluções que respeitam a capacidade dos quartos. Em nenhum dos 40 conjuntos o Gurobi comprovou a otimalidade. Esse comportamento confirma, nas instâncias deste trabalho, a dificuldade dos métodos exatos em instâncias de grande porte relatada na literatura [6, 8], e decorre principalmente do tamanho do modelo exato, que chega a cerca de 28,6 milhões de variáveis na instância muito grande. A heurística, por sua vez, avaliou de 43 a 55 mil soluções por segundo em todos os portes, graças à avaliação incremental, cujo custo depende apenas da internação do paciente movido.

Essas porcentagens comparam os métodos dentro de cada porte e, como uma solução incumbente não convergida serve de referência, medem sobretudo a dificuldade do método exato, e não a qualidade da heurística. Por isso, a qualidade da heurística foi avaliada também por limites inferiores. Além do limite do *solver*, este trabalho utilizou um limite inferior combinatório, obtido diretamente dos dados por uma relaxação na linha da proposta por Ceschia e Schaerf [7], que conta os pacientes-dia que, por falta de leitos da especialidade adequada, ocupam necessariamente quartos de outra especialidade. Esse limite mostrou que

a heurística fica a no máximo 0,3 ponto percentual do mínimo de alocações em especialidade inadequada imposto pelos dados, com desvio padrão de até 0,5 ponto percentual entre os conjuntos. Em outras palavras, quase toda a alocação em especialidade inadequada observada é consequência da distribuição dos quartos, e não da alocação, de modo que reduzi-la exigiria uma decisão estratégica sobre a composição das alas do hospital. Pelo mesmo motivo, a queda do valor de Z por paciente-dia com o porte, de 26,5 para 6,2, se deve principalmente aos dados, já que acompanha a queda do mínimo estrutural. O excedente da heurística sobre esse mínimo, que depende do método, é estável entre os conjuntos e praticamente não varia com o porte, de 3,0 a 3,3 por paciente-dia, mesmo na instância muito grande, em que a heurística dispõe de menos gerações.

A heurística mostrou-se estável, com coeficiente de variação de 1,1% a 1,9% entre execuções de um mesmo conjunto, valor muito menor que a variação entre conjuntos, que chega a 32,3% na instância pequena. A dispersão dos resultados se deve, portanto, principalmente às diferenças entre as instâncias, e não à aleatoriedade do método. As transferências observadas, que chegam a cerca de 15 por 100 pacientes nas instâncias maiores, não indicam uma falha do algoritmo, mas o equilíbrio definido pelos pesos, já que, com $W_{transf}/W_{spec} = 0{,}5$, transferir um paciente para o quarto adequado reduz Z sempre que lhe resta ao menos um dia de internação. O mesmo raciocínio explica parte dos quartos mistos, que a função objetivo prefere quando separar os gêneros exigiria alocar pacientes fora de sua especialidade. A etapa de reparo, além de garantir a viabilidade nas 400 execuções, reduziu o Z médio de 1,2% a 13,7% em relação à mesma heurística sem reparo, com ganho crescente com o porte, principalmente pela redução das transferências.

Conclui-se que o objetivo do trabalho foi atingido. O Algoritmo Memético fornece soluções viáveis e de boa qualidade para o planejamento do censo em todos os portes avaliados. Nas instâncias pequena e média, essa qualidade é certificada por limites inferiores, com a melhor execução a no máximo 8,2% e 19,3% do ótimo, em média, e, nas instâncias maiores, típicas de grandes instituições hospitalares, as soluções são muito superiores às obtidas pelo método exato no mesmo tempo.

5.1 Limitações

A etapa de reparo, que garante a viabilidade das soluções heurísticas, tem custos. Ela consome de 8% a 26% do teto de tempo, conforme o porte, principalmente no reparo dos indivíduos gerados ao acaso, e, ao escolher a realocação mais barata, aumenta os quartos-dia mistos nas instâncias maiores, de 9% a 15% em relação à mesma heurística sem reparo. Além disso, essa comparação com a versão sem reparo foi feita a partir de uma execução anterior do protocolo, e não de execuções pareadas. Essa execução anterior foi realizada com menos processos simultâneos e, por isso, com mais avaliações por execução, o que torna a comparação conservadora.

A comparação entre os métodos também tem ressalvas quanto às condições de execução. O Gurobi e a heurística compartilharam a mesma máquina em processos simultâneos, o tempo de construção do modelo exato não foi contabilizado no teto, o que favorece o *solver*, e as medidas de esforço, iterações do simplex e avaliações da função objetivo, não têm a mesma unidade.

Os limites inferiores disponíveis também são fracos nas instâncias maiores. O limite combinatório considera apenas a parcela de especialidade e, como as transferências e os

quartos mistos respondem por 40% a 50% do Z da heurística nesses portes, permite afirmar apenas que o seu valor é, em média, no máximo cerca de 1,8 a 2,2 vezes o ótimo. A qualidade da heurística é, portanto, certificada de forma satisfatória apenas nas instâncias pequena e média.

O desempenho da heurística é limitado por fatores diferentes conforme o porte. Na instância pequena, a busca estagna por volta da 12ª de cerca de 21 gerações, e o tempo restante não se converte em melhora. Nas demais, o teto de tempo termina com a busca ainda em progresso, com cerca de 10 gerações na instância muito grande.

Por fim, os experimentos foram realizados em instâncias sintéticas e determinísticas, com distribuição de especialidades sorteada para os quartos, o que, na instância pequena, gera conjuntos sem nenhum quarto de alguma especialidade. O peso W_{cap} foi calibrado em um único conjunto da instância média e aplicado a todos os portes, e parte dos testes de calibração foi realizada antes da adoção da etapa de reparo. Critérios presentes na literatura, como faixas etárias, isolamento, carga de trabalho da enfermagem e chegadas de emergência, não foram considerados.

5.2 Trabalhos futuros

As limitações observadas indicam as seguintes direções para trabalhos futuros:

- **Mecanismos contra a estagnação:** nas instâncias pequenas, avaliar estratégias de diversificação acionadas quando a busca deixa de melhorar, de modo a aproveitar as gerações restantes [9].
- **Limites inferiores mais fortes:** desenvolver limites que considerem também as transferências e os quartos mistos, capazes de certificar a qualidade da heurística nas instâncias maiores.
- **Abordagens híbridas com o método exato:** utilizar a solução da heurística como ponto de partida para o Gurobi, ou o Gurobi para resolver subproblemas da heurística, como em abordagens que combinam meta-heurísticas e programação matemática [10].
- **Calibração dos pesos:** avaliar, com gestores hospitalares, a relação entre W_{transf}, W_{gen} e W_{spec}, já que ela determina o número de transferências e de quartos mistos das soluções, e calibrar os parâmetros do algoritmo em mais de um porte e conjunto.
- **Cenários mais realistas:** aplicar o método a dados reais ou a geradores com distribuição de especialidades proporcional à demanda, incluir critérios adicionais, como faixas etárias e isolamento, e estender o modelo para a versão dinâmica do problema, com chegadas de emergência e incerteza no tempo de permanência [6, 2].

Referências Bibliográficas

[1] BRANDEAU, M. L.; SAINFORT, F.; PIERSKALLA, W. P. (Ed.). *Operations research and health care: a handbook of methods and applications*. Boston: Kluwer Academic Publishers, 2004. (International Series in Operations Research & Management Science, v. 70).

[2] SCHÄFER, F. et al. Combining machine learning and optimization for the operational patient-bed assignment problem. *Health Care Management Science*, v. 26, n. 4, p. 785–806, 2023.

[3] HE, L. et al. A systematic review of research design and modeling techniques in inpatient bed management. *Computers & Industrial Engineering*, Elsevier, v. 127, p. 451–466, 2019.

[4] PAIM, J. et al. The Brazilian health system: history, advances, and challenges. *The Lancet*, Elsevier, v. 377, n. 9779, p. 1778–1797, 2011.

[5] BITTENCOURT, R. J.; HORTALE, V. A. Intervenções para solucionar a superlotação nos serviços de emergência hospitalar: uma revisão sistemática. *Cadernos de Saúde Pública*, v. 25, n. 7, p. 1439–1454, 2009.

[6] CESCHIA, S.; SCHAERF, A. Modeling and solving the dynamic patient admission scheduling problem under uncertainty. *Artificial Intelligence in Medicine*, Elsevier, v. 56, n. 3, p. 199–205, 2012.

[7] CESCHIA, S.; SCHAERF, A. Local search and lower bounds for the patient admission scheduling problem. *Computers & Operations Research*, Elsevier, v. 38, n. 10, p. 1452–1463, 2011.

[8] DEMEESTER, P. et al. A hybrid tabu search algorithm for automatically assigning patients to beds. *Artificial Intelligence in Medicine*, Elsevier, v. 48, n. 1, p. 61–70, 2010.

[9] TALBI, E.-G. *Metaheuristics: from design to implementation*. Hoboken: John Wiley & Sons, 2009.

[10] BELIËN, J.; DEMEULEMEESTER, E. Building cyclic master surgery schedules with leveled resulting bed occupancy. *European Journal of Operational Research*, Elsevier, v. 176, n. 2, p. 1185–1204, 2007.

[11] ALA, A.; CHEN, F. Appointment scheduling problem in complexity systems of the healthcare services: A comprehensive review. *Journal of Healthcare Engineering*, Wiley Online Library, v. 2022, n. 1, p. 5819813, 2022.

[12] TAVARES, G. F. *TCC---Hospital-bed-optimization: código-fonte do algoritmo memético e do modelo exato para alocação de leitos*. 2026. Repositório GitHub. Disponível em: <https://github.com/gft109/TCC---Hospital-bed-optimization>. Acesso em: 1 nov. 2026.